Online Class

DSA

Click here to access the online DSA class of 17.10.2026

Click here to access the online DSA class of 31.10.2026

29 July 2026

CSE 203 Code a7z4xque

DSA (Data structure algorithm

CT 4 টা (array থেকে)

CT 1 week 4/5 দিবে (আগে পরে 20 মার্ক)
(searching sorting সহজ)

4 টা CT এর Best 3

Mid এর আগে CT 2

Linked list শেষ হওয়ার পর CT 3

Tree, graph ,, ,, CT 4

Final এ মিলিয়ে সব থাকবে

Introducton to algorithm

THOMAS H. CORMEN

Data s... ...schaum series.pdf

LEC-1 Web view তে open করলে প্রথমের কয় line formatting Powerpoint এ open করা লাগে

LEC-1

Page 1-11

30 July 2026

Slide 12

Linear search

Set k:=1 and LOC:=0

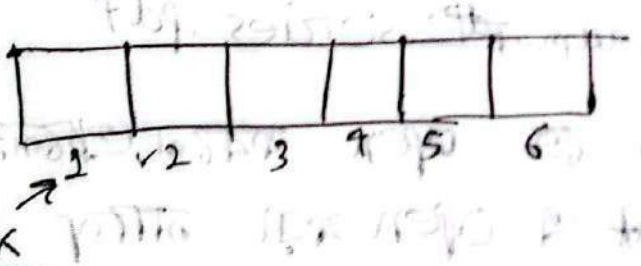
Repeat steps 3 and 4 while k<=N

If Key = L[k] then set LOC:=k, print: Loc and exit

Set k:=k+1

If Loc = 0 the Write: Item is not in list

Exit



location = 0

Binary Search

~~Binary Search~~

1	10	20	30	55	75	190
1	2	3	4	5	6	7

1. key = 55

Beg = 1
End = 7

R_1 (Round 1) → $Mid = \frac{Beg + End}{2} = \frac{1+7}{2} = 4$

L[Mid] < Key
L[4] < 55
30 < 55

→ Beg = Mid + 1 = 4 + 1 = 5

R_2

Beg = 5
End = 7
Mid = 6

L[Mid] > Key
L[6] > 55
75 > 55

End = Mid − 1 = 6 − 1 = 5

R_3

Beg = 5
End = 5
Mid = 5

L[Mid] = key
L[5] = 55
55 = 55

LOC = Mid

~~Bubble sort~~ (Ascending)
Binary search

1. Set LOC := 0, Beg := 1, End := N and
   Mid := (Beg + End) / 2
2. Repeat steps 3 to 6 while Beg <= End
3. If Key < L[Mid], then set End := Mid - 1
4. Else if key > L[Mid] then set Beg := Mid + 1
5. Else if key = List[Mid] then Loc := Mid,
   print : Loc and exit
6. Set Mid := (Beg + End) / 2
7. If Loc = 0 Writ: Item is not in List
8. Exit

Bynary search
~~Bubble sort~~ (decending)

Farhan Hassan

Example: 190 (Beg → 1) 75 (2) 55 (3) 30 (4) 20 (5) 10 (6) 6 (7 → End)

1. Set LOC := 0, Beg := 1, End : N and Mid := (Beg+End)/2
2. Repeat steps 3 to 6 while Beg <= END
3. If Key. < L[Mid], then set Beg = Mid+1
4. Else if key > L[Mid] then set End = Mid-1
5. Else if key = List [Mid] the Loc = Mid
   print : Loc and exit.
6. Set Mid = (Beg + End)/2
7. If Loc = 0 Write: Item is not in List
8. Exit

12 August 2026

O ← big O (O) notation
Ω ← big omega
Θ ← big theta

Complexity of Algorithms

```
int main() { printf("Hello world"); }
```

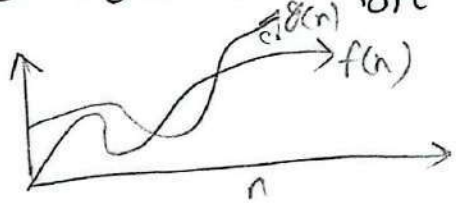
$f(n) = 1$ → constant

```
int main(){
for(int i=0; i<N; i++) printf("Hello");}
```

$f(n) = n$ → linear

Asymptotic Notation

1. O-notation – for the upper bound running time
2. Ω-notation – for the lower bound running time
3. Θ-notation – for the average bound running time



1. O-notation

– A function $f(n) = O(g(n))$ if there are positive constants c and n_0 such that

$f(n) <= c \cdot g(n)$ for all $n >= n_0$

– when $f(n) = O(g(n))$, it is guarantee that f(n) grows at a rate no faster than g(n)
So g(n) is an upper bound on f(n)

মাত্র n এর সাপেক্ষে সবচেয়ে বাকি থাকে focus করে বের করি

$f(n) = O(g(n))$, c = positive constant

iff $f(n) \leq c \cdot g(n)$ for all $n_0 \geq n$

(if and only if)

$f(n) = 5n + 50$

$g(n) = n^1$

(সবচেয়ে সাধারণ)

$f(n) \leq c \cdot g(n)$

$5n + 50 \leq c \cdot n$

~~যদি $c = 1$~~

$5n + 50 \leq 1n$ কখনোই true হয় না

$c = 6$ (bare minimum)

$5n + 50 \leq 6n$

$\Rightarrow 50 \leq 6n - 5n$

$\Rightarrow n \geq 50$

n (number of input)

$n_0 = 50$

n এর value 50 বা তার বড় হলে

$f(n) = O(g(n)) = O(n)$

linear

13 August 2026

Slide

~~Lecture 24~~

1. O- notation (upper bound)

Slide ~~32~~ 31, 32.

33

```
Algorithm Linear Search(a, n, k)
{
    for i := 1 to do    // 0 to n-1        | n+1
        if a[i] = k                        | n
            return i;                      | 1/0
    return -1;                             | 0/1
}
```

$n+1+n+1$

Total step count $= 2n+2$ (Worst case)

$f(n) = 2n+2$

$g(n) = n$

$f(n) \leq c \cdot g(n)$

$\Rightarrow 2n+2 \leq c.n$

$\Rightarrow 2n+2 \leq 3n$

$\therefore n \geq 2 \longrightarrow n_0$

$f(n) = O(g(n)) = O(n)$

Bynary search

1	2	3	4	5	6	7	8
5	6	7	11	15	17	20	21

key = 20

mid

$\frac{n}{2^0}$ $\frac{n}{2^1}$ $\frac{n}{2^2}$ ..... $\frac{n}{2^k}$

$n = 2^k$

$\Rightarrow \log_2 n = k \log_2 2$

$\Rightarrow \log_2 n = k \log_2 2$

$\Rightarrow \log_2 n = k \times 1 = k$

$k = \log_2 8$

$= \log_2 2^3$

$= 3 \log_2 2 = 3 * 1 = 3$

$O(\log_2 n)$

Ans.

19/08/2026

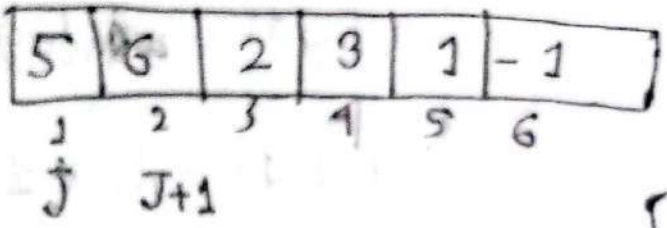
next week বুধবার CT
27 তারিখ ক্লিনিক্যাল আপ্লিকেশনের ক্লাস
পর্যন্ত Mid question-1

Sorting

(Bubble Sort, Insertion Sort, Selection Sort)

Counting sort এই semester এই হবে না
heap sort আর radix sort 6 ক্লাসের

Bubble sort



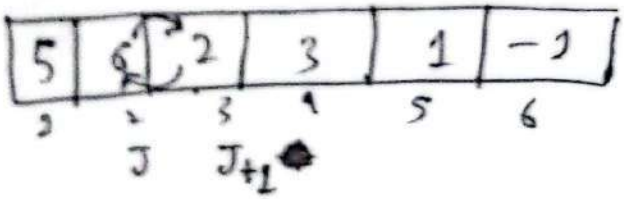
descending হবে < এর A

```
for(i=1; i<N; i++){
    for(j=1; j<N; j++){
        if(a[J]>a[J+1])
            swap(a[J],a[J+1])
    }
}
```

a[J] < a[J+1)

a[1] a[2]

5 6

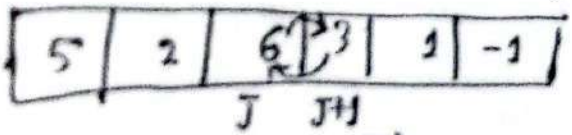


a[J] ≥ a[J+1]

a[1] > a[3]

6 < 2

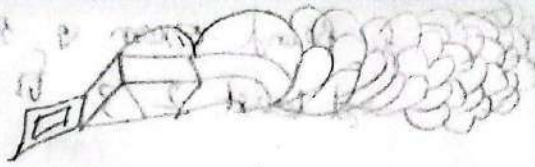
swap(a[J], a[J+1])

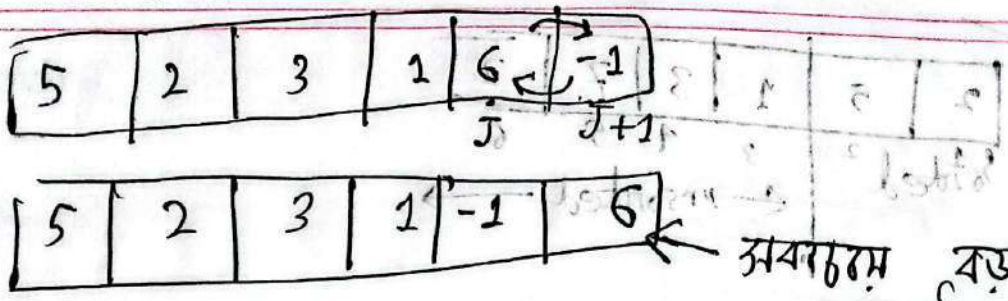


5 2 3 6 1 -1

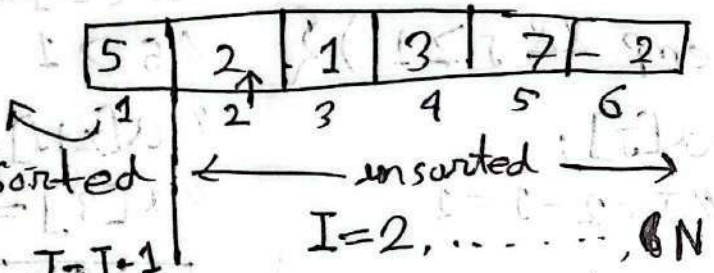
J J+1

বড় bubble সবচেয়ে আগে যাবে
বড় সংখ্যা ~~কম~~ " " Sort হয়।
~~বুদ্ধি~~ নাম bubble sort





~~Selection~~ sort
Insetion



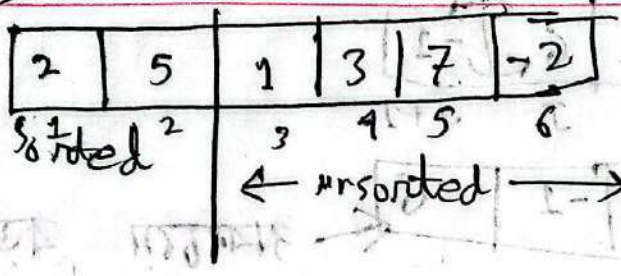
Round 1

I = 2
J = 2 - 1 = 1
~~temp = 2 (I এর location বা value)~~
temp = a[I] = a[2] = 2
while (~~temp~~ arr[J] > temp && J >= 1) {
1. a[J+1] = a[J] // 5 5 (1 2)
2. J = J - 1 // J=1 - 1 = 0
}
//After while loop
a[J+1] = temp
a[1] = 2

2	5

Round exam এ সব না এলেও
at least 2/3 টা Round নিখুঁত

Round 2



I = 3

J = 3-1 = 2

temp = a[I] = a[3] = 1

while(a[J] > temp && J >= 1){

a[J+1] = a[J];

J = J-1 // J = 2-1 = 1

}

a[J+1] = temp

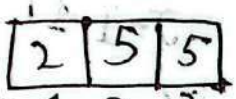
//a[1] = temp

//a[2]

//5 > 1

// a[J+1] = a[2]

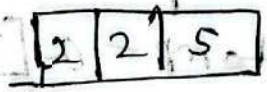
//a[3] = 5

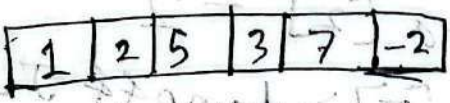


a[1] (2 > 5)

a[1+1]

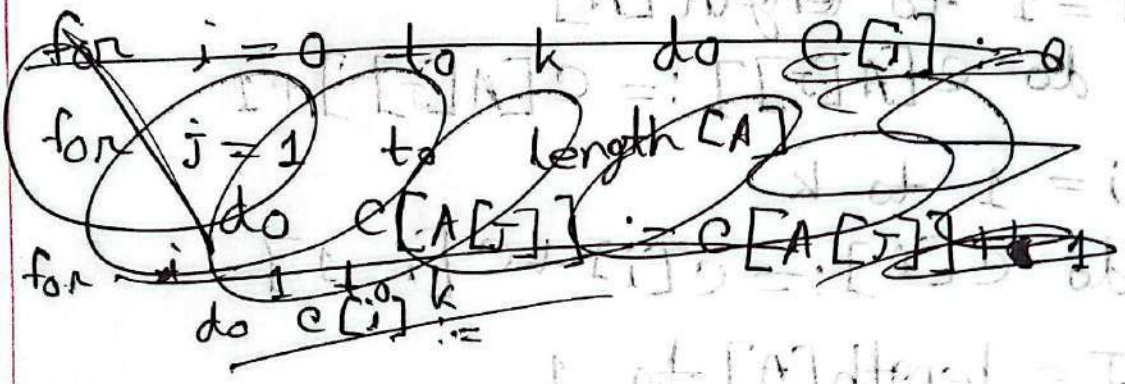
a[2] = 2





20/08/2026

Counting_Sort (A, B, C, k)



A[8] = | 2 | 5 | 3 | 0 | 2 | 3 | 0 | 3 |
1 2 3 4 5 6 7 8

(0, 1, 2, 3, 4, 5)

k = max − min + 1 = 5 − 0 + 1 = 5 + 1 = 6

C[6] = | 0 | 0 | 0 | 0 | 0 | 0 |
0 1 2 3 4 5

i

minimum value 0 of position

C[i] = 0

C[0] = 0; C[1] = 0 ... C[5] = 0

C[6] = | 0 | 0 | 0 | 0 | 0 | 0 |
0 1 2 3 4 5

C[A[j]] = C[A[j]] + 1

j = 1

C[A[1]] = C[A[1]] + 1

C[2] = C[2] + 1

= 0 + 1

= 1

min ↓ max ↓

for $i = 0$ to k do $C[i] := 0$

for $J = 1$ to length $[A]$
do $C[A[J]] := C[A[J]] + 1$

for $i = 1$ to k
do $C[i] := C[i] + C[i-1]$

for $J =$ length $[A]$ to 1
do $B[C[A[J]]] := A[J]$
$C[A[J]] := C[A[J]] - 1$

দৃষ্টান্ত

~~$C[A[J]] = C[A[J]] + 1$~~

second loop এর পর

$C[6] =$

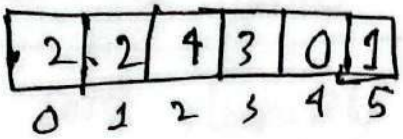
2	0	2	3	0	1
0	1	2	3	4	5

$C[i] = C[i] + C[i-1]$

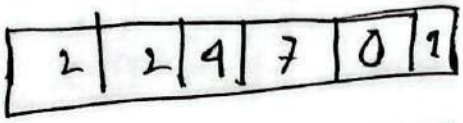
$i=1 \Rightarrow C[1] = C[1] + C[1-1] = 0 + C[0] = 0 + 2 = 2$

2	2	2	3	0	1
0	1	2	3	4	5

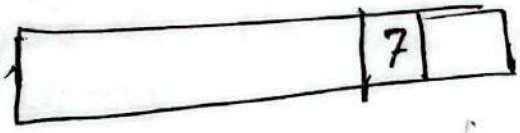
j=2: $C[2] = C[2] + C[1] = 2 + 2 = 4$



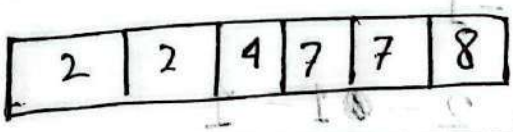
j = 3



j = 4

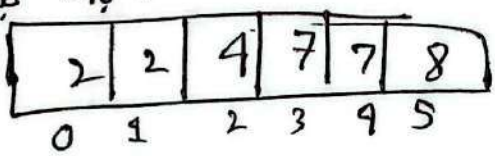


j = 5

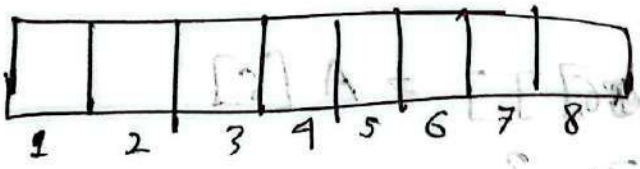


3rd loop শেষ হয়।

$C[6] =$



$B[8] =$



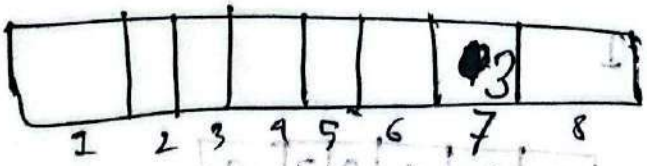
j=8 $B[C[A[j]]] = A[j]$

$\Rightarrow B[C[A[8]]] = A[8]$

$\Rightarrow B[C[3]] = 3$

$\Rightarrow B[7] = 3$

$B[8] =$



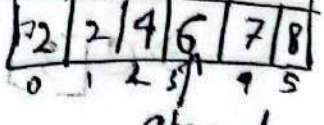
j = 8

$C[A[8]] = C[A[8]] - 1$

$\Rightarrow C[3] = C[3] - 1$

$= 7 - 1 = 6$

$C[8] =$



j=7 $B[c[A[7]]] = A[7]$

$\Rightarrow B[c[0]] = 0$

$\Rightarrow B[2] = 0$

B

1	2	3	4	5	6	7	8
	0					3	

J=7

$c[A[7]] = c[A[7]] - 1$

$c[0] = c[0] - 1 = 2 - 1 = 1$

c

1	2	3	4	5
1	2	4	6	7 8

J=6 $B[c[A[6]]] = A[6]$

$\Rightarrow B[c[3]] = 3$

$\Rightarrow B[6] = 3$

B

1	2	3	4	5	6	7	8
	0				3		8

$c[A[6]] = c[A[6]] - 1$

$\Rightarrow c[3] = c[3] - 1$

$= 6 - 1$

$= 5$

c

1	2	3	4	5	6
1	2	4	5	7	8

J এর Value A এর index
A " " C " "
C " " B " "
B " " =A এর Value
C এর Value 1 minus

Simulation করতে বললে Algorithm লেখা নিরুত্তর হবে। Algorithm লেখার দরকার নাই

Counting sort stability Maintain slide এর ফুল k=5 করবা করতে হবে
Counting sort Sequance " " "
time complexity যদি ডিরেক্ট বলে যাবে

Senerio এর depend করে Sort Algorithm pick করা লাগবে।

CT এ চায় sort
Cselection sort করাতে পারি নাই তাই CT তে নাই)

next week এ লাস্ট ক্লাসের CT (সিলেবাস Sorting, searching realated minimum এগুলো)

2/09/2026

5	2	3	1	7
1	2	3	4	5

Insertion Sort

i ↑ j

min = i = 1

j = i+1 = 1+1 = 2

if(A[min] > A[j])

A[1] > A[2]

5 > 2

min = j = 2

j = i+1 = 2+1 = 3

if(A[min] > A[j])

A[2] > A[3]

2 ≯ 3

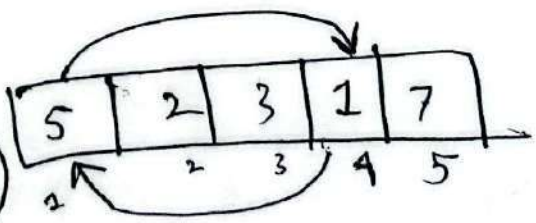
~~min = j~~

~~j = 3+1 = 4~~ (2 থেকে 5 পর্যন্ত)

Loop চলে এবং সর্বনিম্ন অবস্থান 1 এর index এগুলো min এ

⇒ min = 4

swap(A[min], A[i])



Radix Sort না পড়লেও Exam তে নাই।

After Round 1

1	2	3	5	7

Next Round শুরুতেও Loop চলবে.

আবার Loop N times

ভিতরেও Loop তে প্রায় N times

তাই Worst case তে N×N times

∴ Time complexity $O(N^2)$ (Nested loop এলে মূলত 2nd টা)

Selection, Insertion, Bubble sort এর $O(N^2)$

Counting Sort

$k = max - min + 1$

C[k]

A, B → N

C → k

```
for (i = 1 ---- k)  ---> N times
for (i = 0 ---- k)  ---> k times
```

k = 100

যদি N = 100

$O(N+k) = O(100+100) = O(200)$

$O(N^2) = O(10000)$

যদি N = 1000

$O(N+k) = O(1100)$

$O(N^2) = O(1000000)$

Time complexity $O(N+k)$

★ ADT কী

Stack

Abstract data type

ADT কী

Abstract — নিজের structure নাই।
different data type কে help করার

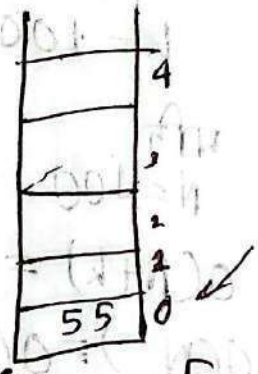
LIFO — Last In First Out

Stack LIFO concept এর উপর করে।

Special Variable **TOP**

primary operator দুইটি ~~Push → Pop~~

push — insert
pop — delete



Algorithm Based এ 1 থেকে শুরু
কিন্তু Stack এ শুরু 0

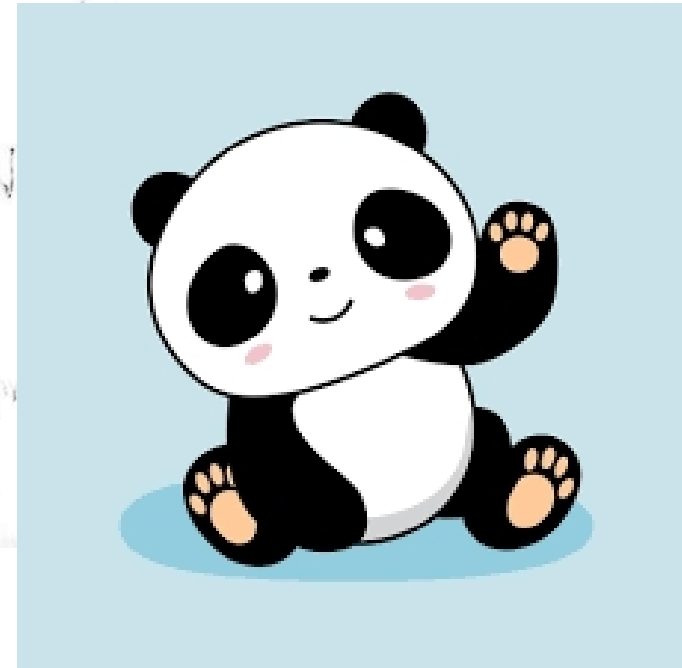
Size = 5

Top = -1

Push (value)

Top++ → 0

Stack [Top] = value
0 55

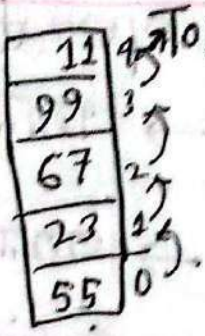


Push 23

Top ++ → 1

stack [TOP] = 23

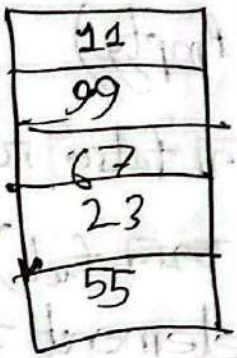
1



Top == Size - 1

→ Push করলে overflow

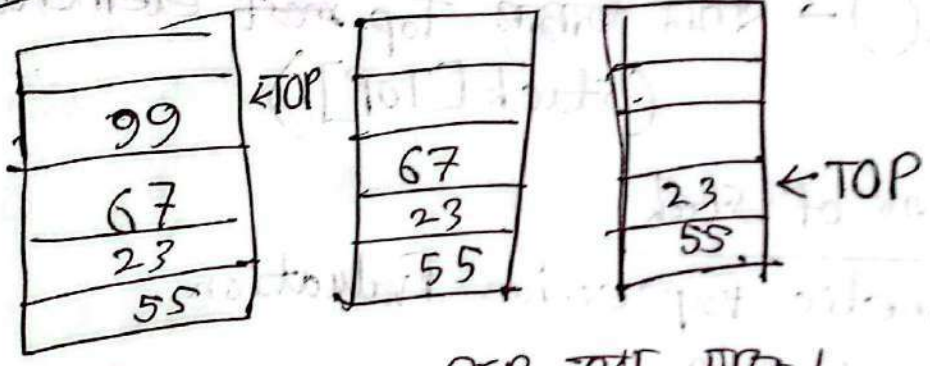
POP() Example



TOP আছে যে

stack [TOP] = NULL

Top --



5 টা ডিস্ক। 5 বার POP করা যাবে।



If (Top = -1) Underflow

Stack Example: Text editor এর undo, ~~redo~~

Secondary operations — (Index)

1. TOP() → এখন TOP কোথায় আছে তা return করে
2. size() → এখন আমার size কত (size = Top+1) (কয়টি সংখ্যা আছে)
3. isEmpty() → Empty হলে True নাহলে false return (যখন Top = -1 তখন Empty)
4. isFull() → Full হলে True নাহলে false return (যখন Top = size-1 তখন full)
5. peek() → এখন আমার Top. ~~not~~ element কত (stack[Top])

Applications of Stack

- Arithmetic Expression Evaluation
- Syntax Parsing
- Solving search Problem

Arithmetic Expression

Three types of arithmetic expression:

1: Infix Expression

2 + 4, a − b/c

2: Prefix expression

+ 2 4, − a/b c

3: Postfix Expression

2 4 +, a b c / −

Infix → Prefix

2. F = (5 + 7) * 8 − 10

step 1: 12

step 2: 12 * 8 − 10
96

step 3: 86

Infix এ 3 বার Round ঘুরা লাগলে Computer এ অনেক বড় Problem

~~Parenthesis balanced~~ ~~কিনা~~ ~~চেক~~ ~~করার~~ ~~Lab task~~ ~~ছিল~~

1 (carret)

Infix → Postfix

5 * (6+2) - 12/4

(প্রথমেই আমরা stack এ Top এ push

Infix	stack	Postfix
5		5
*	* (0 1 2 3 4)	5
(	* ((0 1)	5
6	* (	5,6
+	* (+ (0 1 2)	5,6
2	* (+ (0 1 2)	5,6,2
)	* (0 1)	5,6,2,+
-	-	5,6,2,+,*
12	-	5,6,2,+,*,12
/	- /	5,6,2,+,*,12
4	- /	5,6,2,+,*,12,4

Empty stack এ আসার problem হয়

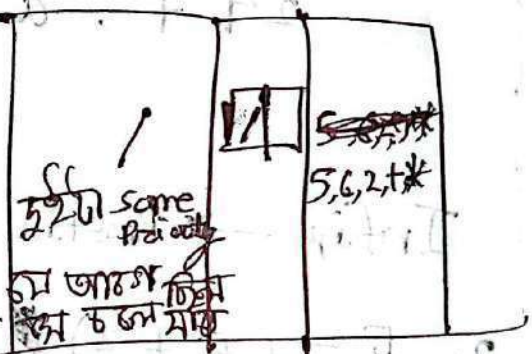
(আসার পর first আর push

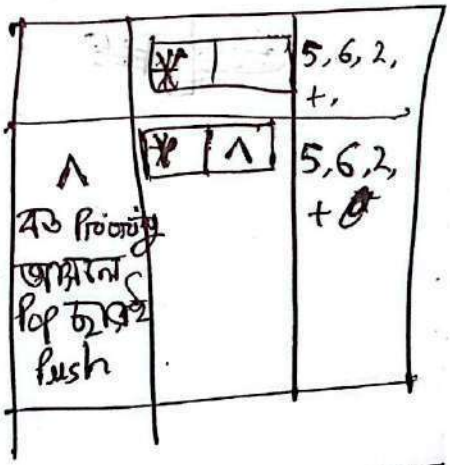
+ কে pop করে postfix এ নিয়ে (কে pop হয়ে চলে যাবে

stack খালি না তাই এর বড় কোন আগের * এর priority বেশী * চলে যাবে (pop) - আসবে।

/ এর priority বেশী powerful

25 নং slide next দিন নিয়ে বসতে দিবে। 1 মার্কস Free





expression খালি stack 1 by 1 pop

5, 6, 2, +, *, 12, 4, /, -

1. 8 * (5 ^ 4 + 2) - 6 ^ 2 / (9 * 3)

Infix expression	stack	Postfix Expression
8		8
*	*	8
(	* (	8
5	* (	8 5
^	* (^	8 5
4	* (^	8 5 4
+	* (+	8 5 4 ^
2	* (+	8 5 4 ^ 2
)	*	8 5 4 ^ 2 +
-	-	8 5 4 ^ 2 + *
6	-	8 5 4 ^ 2 + * 6
^	- ^	8 5 4 ^ 2 + * 6
2	- ^	8 5 4 ^ 2 + * 6 2
/	- /	8 5 4 ^ 2 + * 6 2 ^
(	- / (	8 5 4 ^ 2 + * 6 2 ^
9	- / (	8 5 4 ^ 2 + * 6 2 ^ 9
*	- / (*	8 5 4 ^ 2 + * 6 2 ^ 9
3	- / (*	8 5 4 ^ 2 + * 6 2 ^ 9 3
)	- /	8 5 4 ^ 2 + * 6 2 ^ 9 3 *
		8 5 4 ^ 2 + * 6 2 ^ 9 3 * / -

2. A + (B * C - (D / E ^ F) * G) * H

Infix expression	stack	Postfix expression
A		A
+	+	A
(	+ (	A
B	+ (	A B
*	+ (*	A B
C	+ (*	A B C
-	+ (-	A B C *
(	+ (- (	A B C *
D	+ (- (	A B C * D
/	+ (- (/	A B C * D
E	+ (- (/	A B C * D E
^	+ (- (/ ^	A B C * D E
F	+ (- (/ ^	A B C * D E F
)	+ (-	A B C * D E F ^ /
*	+ (- *	A B C * D E F ^ /
G	+ (- *	A B C * D E F ^ / G
)	+	A B C * D E F ^ / G * -
*	+ *	A B C * D E F ^ / G * -
H	+ *	A B C * D E F ^ / G * - H
		A B C * D E F ^ / G * - H * +

Post-fix Calculation ৭ প্রতি Iteration এ হয় যায়

3/9/26 Post-fix to value

Example: 5, 6, 2, +, *, 12, 4, /, -

Postfix expression	Stack
5	5
6	5 6
2	5 6 2
+	5 8
*	40
12	40 12
4	40, 12 4
/	40 3
-	37

Sign পাওয়া মাত্র pop হয়ে যায়

Pop এর সময়
A = 2
B = 6
B ϕ A
6 + 2
= 8

Phi (ϕ) দ্বারা operation

(-) বা (/) এর সময় মাঝে গড়বিল না হয়। তাই B ϕ A হয়।

operators	precedence
( )	Highest
^	Next highest
*, /, %	Next ^ highest
+, -	Lowest

^
/*
+ -

1. Evaluate

8 5 4 ^ 2 + * 6 2 ^ 9 3 * / -

Prefix expression	Stanck
8	8
5	8 5
4	8 5 4
^	8 625
2	8 625 2
+	8, 627
*	5016
6	5016 6
2	5016 6 2
^	5016 36
9	5016 36 9
*	5016 324
/	5016 15.18
3	5016 36 9 3
*	5016 36 27
/	5016 1.33333333
-	5014.666666666666666667

9/9/25

Linear queue এর problem
এই queue সমস্যা bad overflow

Queue
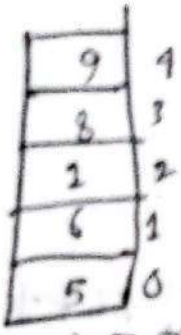
ADT

Array/LinkedList দিয়ে করা যায়

Linear queue

1. Insert (Enqueue)
2. Delete (Dequeue)

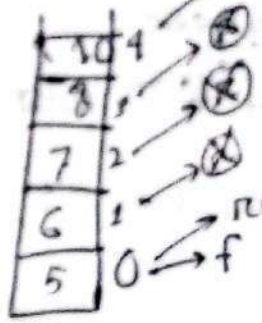
FIFO list



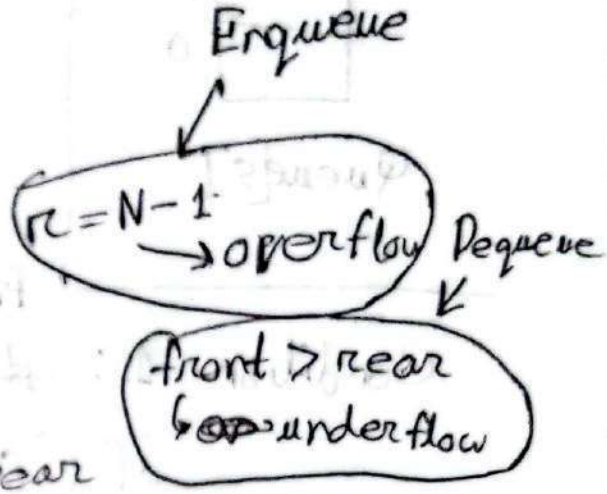
Stack[5]

Top = -1

Push() → Top++
stack[Top] = value



Queue[5]
rear = -1
front = -1



(insert)
(delete)

Insert (Enqueue)

```
if (front == -1 && rear == -1)
    front = 0;
rear++
queue[rear] = value
```

Stack এর Queue এর Insert almost same

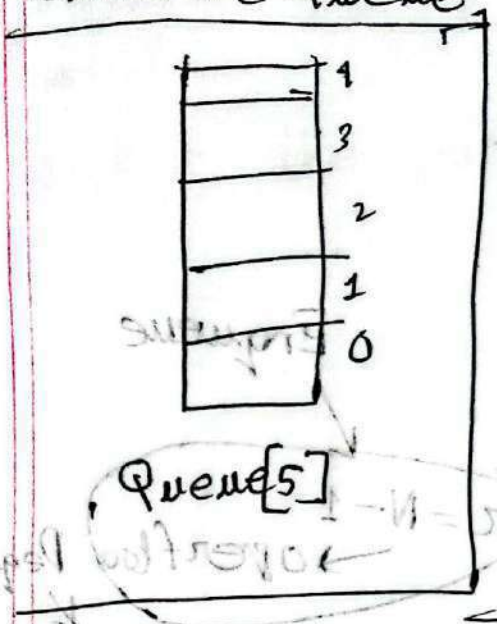
Dequeue (Delete)

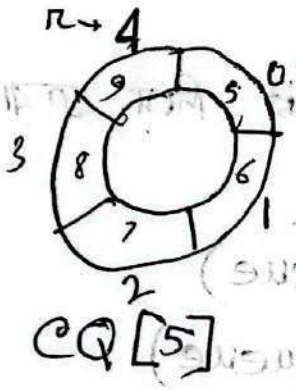
front++ (condition উল্টে আসবে)

"Reuse" নিয়ে question আসবে

random access না support করবে, Insertion & deletion can be done from different ends (Queue)

Circular Queue





Enqueue

Condition 1:

```
if (f = r = -1) → f = 0
```

Condior 2:

```
if ((r == N-1 && f == 0) || r+1 == f)
    → overflow
```

Contion 3:

```
if (r == N-1)
    → r = 0
```

Condition 4:

```
CQ[r] = value
```

```
r++
CQ[r] = value
```

Dequeue ()

Condition 1:

if(f > r)
↳ underflow

Condition 2:

if (f = r = N-1)
f = 0

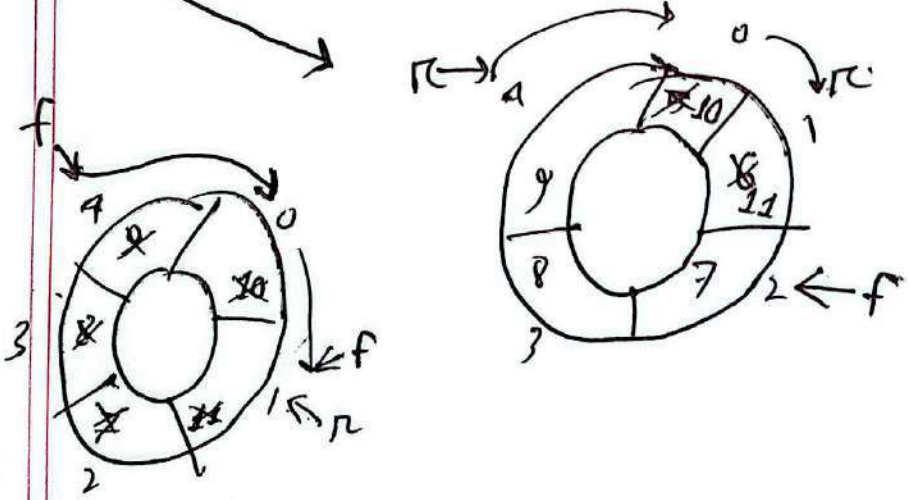
Condition 3:

if(r == f)
f = -1
r = -1

optional
(যেন Queue খালি তাই reset করা।
না দিলেও মার্ক করতে কোন সমস্যা নেই।

Condition 4:

CQ[f] = NULL
f++



এইবারের Mid এ দুটো স্পেশালি প্রশ্ন করবে।
most recent. term টা মানেই LIFO (stack) হবে।

Fall 25 Mid question এ আসবে।



Department of Computer Science & Engineering
University of Asia Pacific (UAP)

Class Test #1 **Spring 2026** **2nd Year 1st Semester**

Course Code: CSE 203 **Course Title: Data Structures and Algorithms I** **Credits: 3**

Full Marks: 10 **Duration: 30 minutes**

Instructions:

1. Part marks are shown in the margins.
2. Non-programmable calculators are allowed.

Name:	ID:

1. Linear search normally examines elements sequentially until the target is found. Suppose an unsorted array contains duplicate values, and you need to determine the first occurrence, last occurrence, and total number of occurrences of a given element. [4]

   Explain how linear search can be modified to accomplish all three tasks in a single traversal of the array.

2. Below is the pseudocode for counting sort. There are some errors in it.

```
Counting_Sort( A, B, C, k, N)
 i.   for j = 1 to N
ii.       do C[A[j]] := C[A[j]]+1
.ii.  for i = 1 to N
iv.       do C[i] := C[i]+ C[i-1]
 v.   for j = N to 1
vi.       do C[A[j]] := C[A[j]]-1
'ii.      A[C[B[j]]] := A[j]
.ii.  End.
```

   a. Find the errors. Explain why and how the above-stated code would produce wrong results using one example. [3]

   b. Now correct the pseudocode. [3]

pseudocode

1. Linear search ~~algorithm~~ is given below which can accomplish all three tasks-

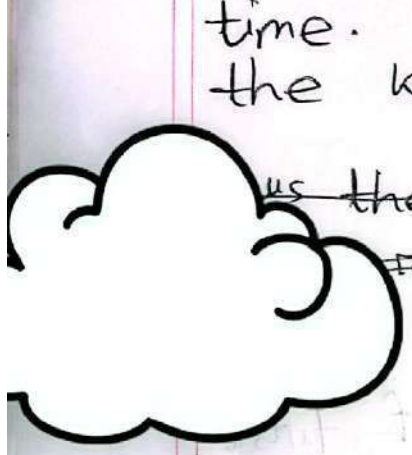
Linear_Search (Array, N, key, first, last, count)

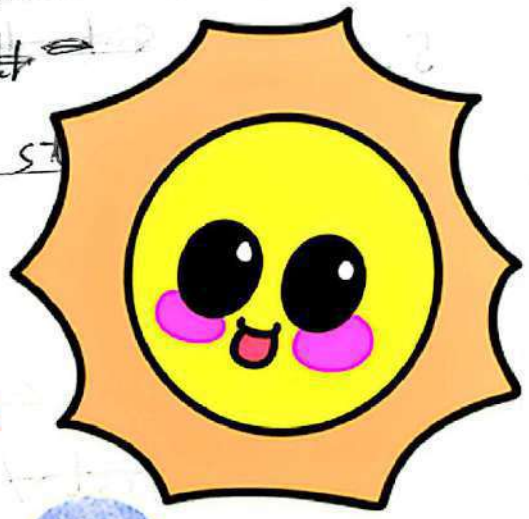
1. set first := -1, last := -1 and count := 0
2. for i = 1 to N do:
3. If Array[i] = key then
4. ~~set first = i~~
   If first = -1 then
5. set first := i
6. set last := i
7. Set count := count + 1
   ~~If first ≠ -1 then Print the value of~~
   ~~If first ≠~~
8. If count > 0 then
9. print "First occurance Position: ", first
10. print "Last occurance Position: ", last
11. print "Total occurance: ", count
12. Else
13. print "Element not found in array
14. End

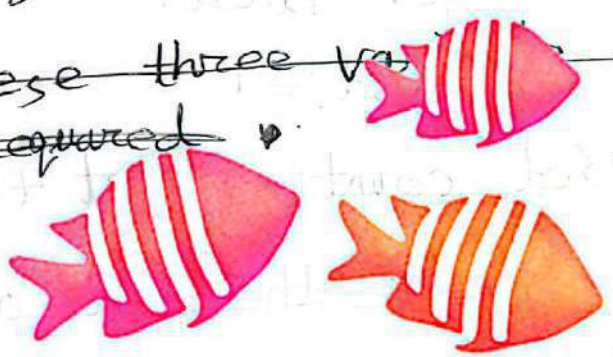
Here three variables (first, last, count) are used to perform the three task. At the begenning first and last are set to -1 and count is set to 0. While loping throug array It is

the element matches the ~~value~~ key that need to be searched. If Its matched the first is updated ~~to~~ as that position if first is -1. If first is not -1 the it was already updated before so it only updates once. The last is updated whenever key is matched so after the last occurance it updates for the last time. count is incremented whenever the key matched. ~~Does this~~

~~as these three va~~ ~~st~~
~~required~~.









আমি ছোট করে লিখতে পারিনা (৩ পৃষ্ঠা) 😊

2(a) There are four errors-

1. Initialization missing:

Array C was suppose to be set to 0 at the begenning. But the loop isn't there. As a result C will hold garbage value that ~~will give wrong ans~~ can cause out-of-bound error.

2. Incorrect loop limit in line (v)

The third loop was supposed to loop from 1 to K which is looping from 1 to N.

3. At line (vi) C[A[J]] is decremented before placing element in the output array. Line (vi) and (vii) need to be exchanged

4. At line (vii) A array is updated instead of array B.

Example:

Let an array a[5] = {2, 5, 4, 1, 0} here k = 5-0+1 = 6

Since B isn't intialized it will hold garbage value. For simplicity lets assume with garbage values B[k] = B[6] = {5, 10, 0, 3, 5, 2}

~~After finish loop 4 Array C will be~~

In the first loop

$j = 1 \rightarrow c[A[1]] = c[A[1]] + 1$

$\Rightarrow c[2] = c[2] + 1 = 0 + 1 = 1$

	0	1	2	3	4	5
C =	5	10	10	3	5	2

$J = 2 \rightarrow c[5] = c[5] + 1 = 2 + 1 = 3$

C =	5	10	01	3	6	3

Similarly -

$J = 3 \rightarrow$ C = | 5 | 10 | 01 | 3 | 6 | 3 |

$J = 4 \rightarrow$ C = | 5 | 11 | 01 | 3 | 6 | 3 |

$J = 5 \rightarrow$ C = | 6 | 11 | 1 | 3 | 6 | 3 |

In the second loop -

$i = 1 \rightarrow C[1] = C[1] + C[0] = 11 + 6 = 17$ ~~$10 + 5 = 15$~~

C = | 6 | 17 | 1 | 3 | 6 | 3 |

similary

$i = 2 \rightarrow$ C = | 6 | 17 | 18 | 3 | 6 | 3 |

$i = 3 \rightarrow$ C = | 6 | 17 | 18 | 21 | 6 | 3 |

$i = 4 \rightarrow$ C = | 6 | 17 | 18 | 21 | 27 | 3 |

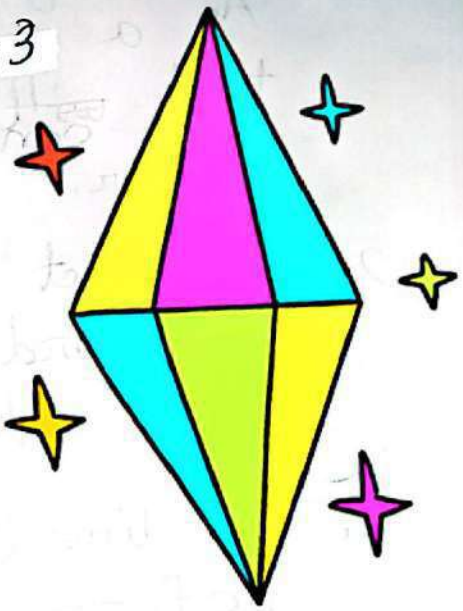
$i = 5 \rightarrow$ C = | 6 | 17 | 18 | 21 | 27 | 30 |

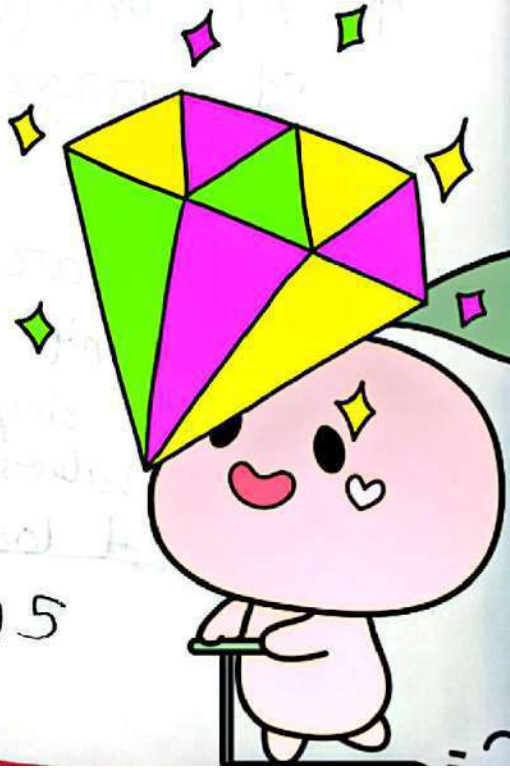
In the third loop

$J = 5 \rightarrow C[A[5]] = C[A[5]] - 1$

$\Rightarrow C[0] = C[0] - 1 = 6 - 1 = 5$

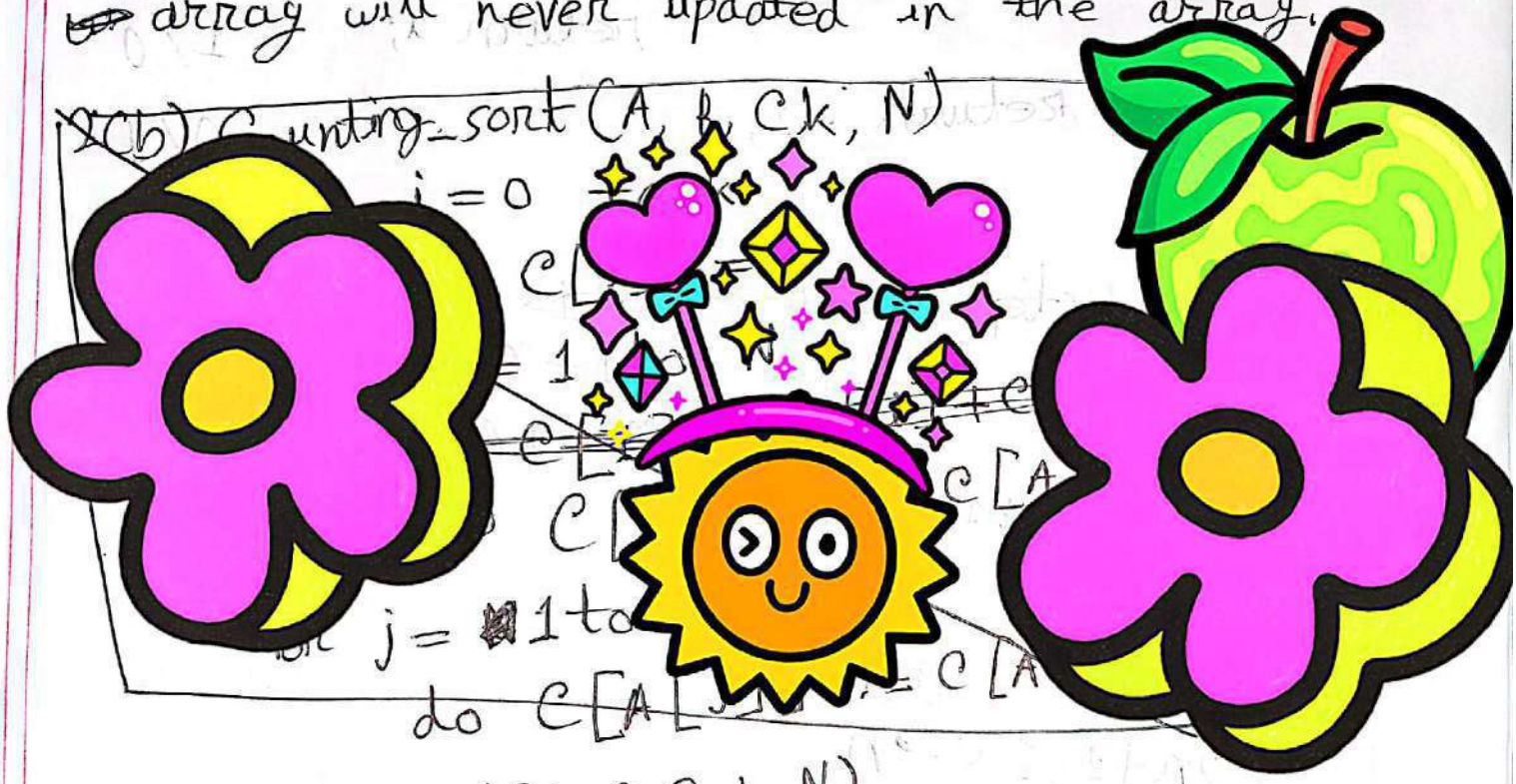
C = | 5 | 17 | 18 | 21 | 27 | 30 |





and A[C[B[5]] = A[5]

Here A is updated and B holds a garbage value. B is the array which is suppose to hold the sorted array. Thus the requared sorted array will never updated in the array.



2(b) Counting_Sort(A, B, C, k, N)

```
for i = 0 to k
    do C[i] := 0
for j = 1 to N
    do C[A[j]] := C[A[j]] + 1
for i = 1 to k
    do C[i] := C[i] + C[i-1]
for j = N to 1
    do B[C[A[j]]] := A[j]
    C[A[j]] := C[A[j]] - 1
End
```

Department of Computer Science & Engineering
University of Asia Pacific (UAP)

Class Test #1 **Spring 2026** **2nd Year 1st Semester**

Course Code: CSE 203 **Course Title: Data Structures and Algorithms I** **Credits: 3.0**

Full Marks: 10 **Duration: 30 minutes**

Instructions:

Part marks are shown in the margins.

Non-programmable calculators are allowed

Name: **ID:**

1. Why is Binary Search more efficient than Linear Search for large inputs? Derive the time complexities for both algorithms and then prove your case. [4]

2. Below is the algorithm for binary search. There are some errors in it.

Algorithm: Binary-Search (L, N, Key)

1. Set Loc := -1, Beg := 0, End :=N
2. Repeat steps 3 to 6 while Beg <= End
3. Mid := (Beg + End)/2
4. If Key < L[Mid], then Set End := Mid
5. Else if Key>L[Mid] then Set Beg := Mid +1
6. Else if Key = Mid then Loc:= Key, print: Loc.
7. If Loc = 0 Write: Item is not in List.
8. Exit.

i. *Find the errors. Take 5 random integers that are sorted in ascending order and a key value of your choice. Explain why and how the above-stated code would produce wrong results using your example.* [4]

ii. Now correct the pseudocode.

CamScanner

1. Linear_Search(a, n, k)

```
{
    for i := 1 to n do          n+1
        If a[i] = k             n
            return i;           1/0
    return -1;                  0/1
}
```

Total step count
$= n+1+n+1+0$
$= 2n+1$ (Worst Case)

$f(n) = 2n+1$
$g(n) = n$

Now, $f(n) \leq c \cdot g(n)$
$\Rightarrow 2n+1 \leq c \cdot n$

The inquity cant be true unless c is at least 3

$\therefore 2n+1 \leq 3n$
$\Rightarrow 3n-2n \leq 1$
$\Rightarrow n \leq -2$
$\Rightarrow n \geq 2$

$\therefore$ if n is greater than 2 than
$f(n) = O(g(n)) = O(n)$

In binary search it ~~skips half~~ decreases half of the remaing search space in every step until it reaches 1.

∴ The search space after every step

$$\frac{n}{2^0}, \frac{n}{2^1}, \frac{n}{2^2}, \frac{n}{2^3}, \ldots\ldots, \frac{n}{2^k}$$

here k is the ~~total~~ maximum number of steps



here

$$\frac{n}{2^k} = 1$$

$$\Rightarrow n = 2^k$$

$$\Rightarrow \log_2 n = \log_2 2^k$$

$$\Rightarrow \log_2 n = k(\log_2 2)$$

$$\Rightarrow \log_2 n = k$$





∴ Time complexity is $O(\log_2 n)$

For a large dataset of size $n = 1000000$

Linear search requires $n = 1000000$ comparisons

For binary search it requires $\log_2 1000000 = 20$ comparisons

here $20 < 1000000$

Therefore Binary search is more efficient than linear search for large inputs.

CamScanner

2. (i) Errors:-

1. At line ① ~~Location~~ Beg need to set as 1 instead of 0. Loc is set to -1, which should be set to 0. ~~since at line~~ Since it compared with 0 at the end.
2. At line 4 End need to set as Mid - 1. Setting it to Mid can cause infinite loop.
3. In line 6 Key is compared with Mid. Mid holds index. So Key ~~should~~ ~~co~~ need to compare with L[Mid]
4. In line 6 Loc is set to Key instead of Mid

Example

L[5] = {10, 20, 30, 40, 50}, Let key = 10

Loc = -1, Beg = 0, End = N = 5

Iteration 1 -

$Mid = \frac{0+5}{2} = 2$

L[Mid] = L[2] = 20

10 < 20 is True. Therefore End = Mid = 2

Iteration 2 -

$Mid = \frac{0+2}{2} = 1$

L[Mid] = L[1] = 10

~~10 = 10 is true. Therefore~~ 10 is not greater or less than 10

So At line 6 it compares 10 = 1 which is false

~~So no~~

Iteration 3 –

$$Mid = \frac{0+1}{2} = 0$$

Here L[Mid] = L[0] is out-of-bound

2(ii)

1. Set Loc := 0, Beg := 1, End := N
2. Repeat steps 3 to 6 while Beg <= End
3. Mid := (Beg + End)/2
4. If Key < L[Mid], then set End := Mid - 1
5. Else if Key > L[Mid], then Set Beg := Mid + 1
6. Else if (Key = L[Mid]), then set Loc := Key, print: Loc.
7. If Loc = 0 Write: Item is not in List
8. Exit







CamScanner

23/9/2026

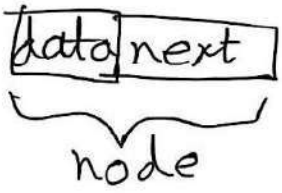
Final ৭ minimum 10 mark ~~15~~ থাকবে 15 linked list

24/9/2026

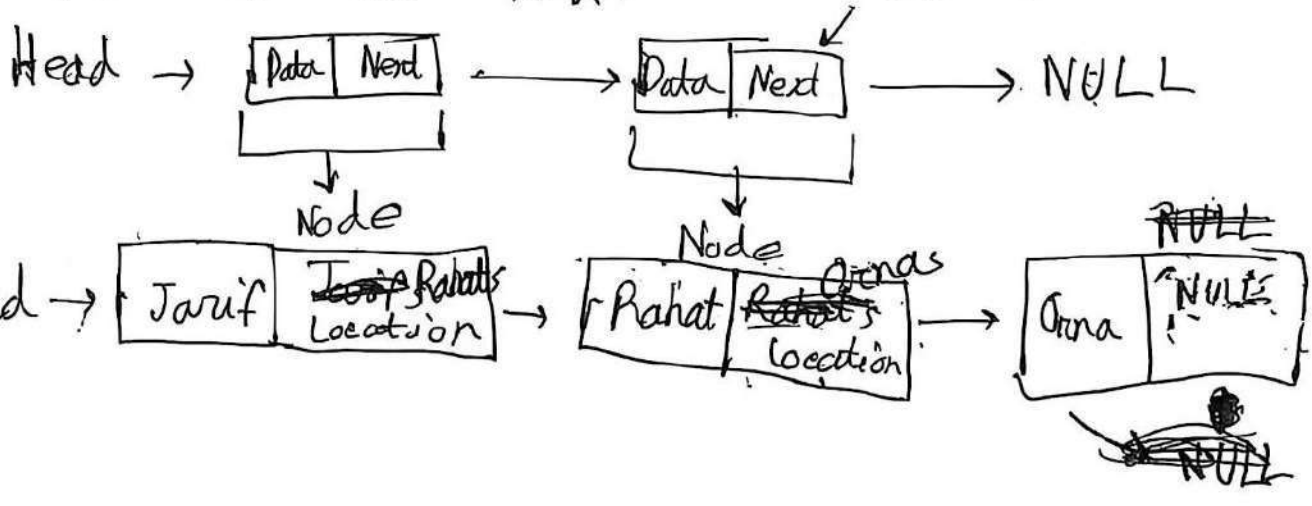
Linked List

Linear data structure

- A linked list is an ordered collection of finite, homogeneous data elements called nodes where the linear order is maintained by means of links or pointers.
- Each element is represented by a node



- Each nod is divided into two parts:
  1. Data part contains the information of element
  2. Link part contains the address of the next node in the link.



- Linked list has a pointer variable, Head or start containg address of first node
- If start contains NULL value, it means the list is empty
- The last node contains NULL reprosting invalid

Dynamic representation

Terminologies

Malloc()

```
#include <stdlib.h>
int* ptr = (int*) malloc (5*sizeof(int));
```

(int*) optional

4 byte

ptr = 20 bytes of memory

Structure

```
struct node
{
    int data;
    struct node *next;
};
```

Int ptr; Pointer

self refrence

5000 → 5011

-> arrow operator

ptr = malloc (sizeof (struct node))

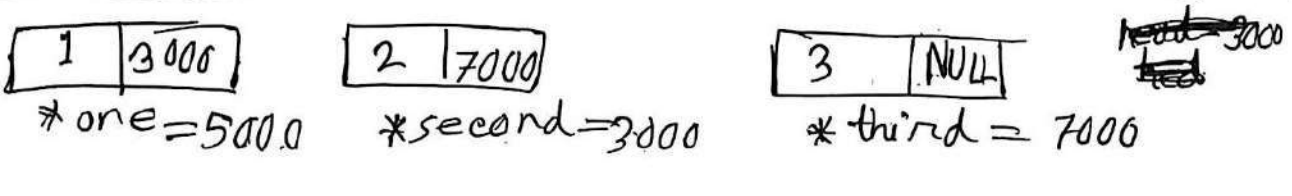
```
struct node *head;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

one = malloc (sizeof (struct node));
two = "";
three = ""

one->data = 1;
two->data = 2;
three->data = 3;

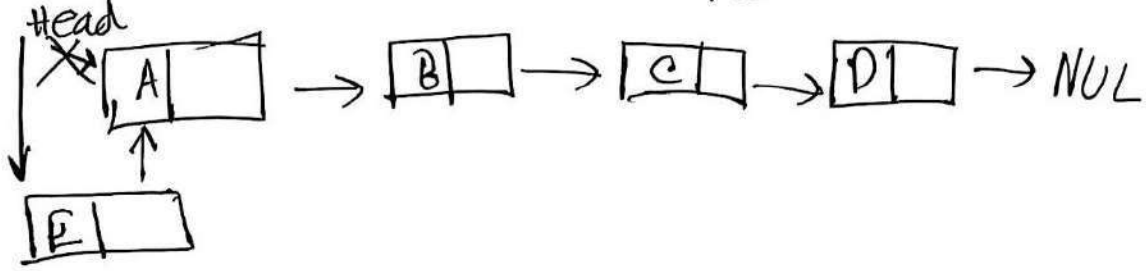
one->next = two;
two->next = three;
three->next = NULL;

head = one;
```



34/09/2026

Add a new node at front



Algorithm: InsertAtHead (head, data)

1. Create a new node NEW
2. Set NEW → data = data
3. set NEW → next = head
4. Update head = NEW
5. Return head

Time complexity : O(1)

* head = NULL

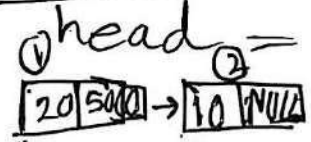


*new = malloc( ) 5000

new → data = value;
10

new → next = head

head = new

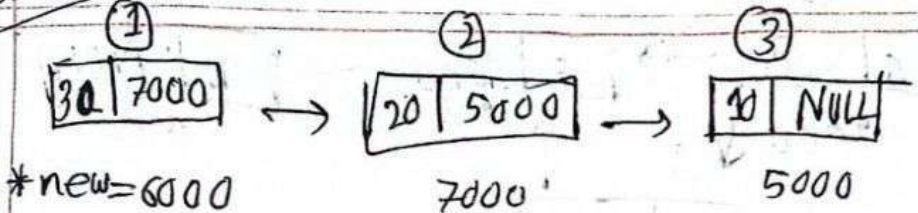


*new = 7800

new → data = value = 20

new → next = head = 5000 head = new

3rd time calling

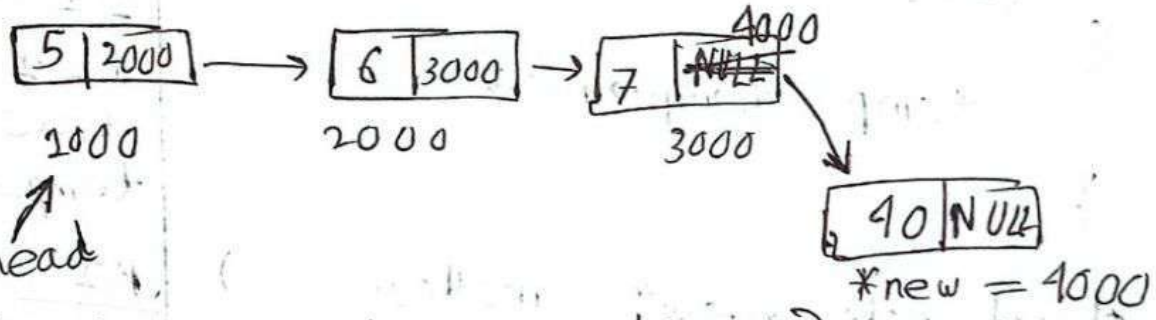


new → data = value = 30

new → next = head = 7000

head = new

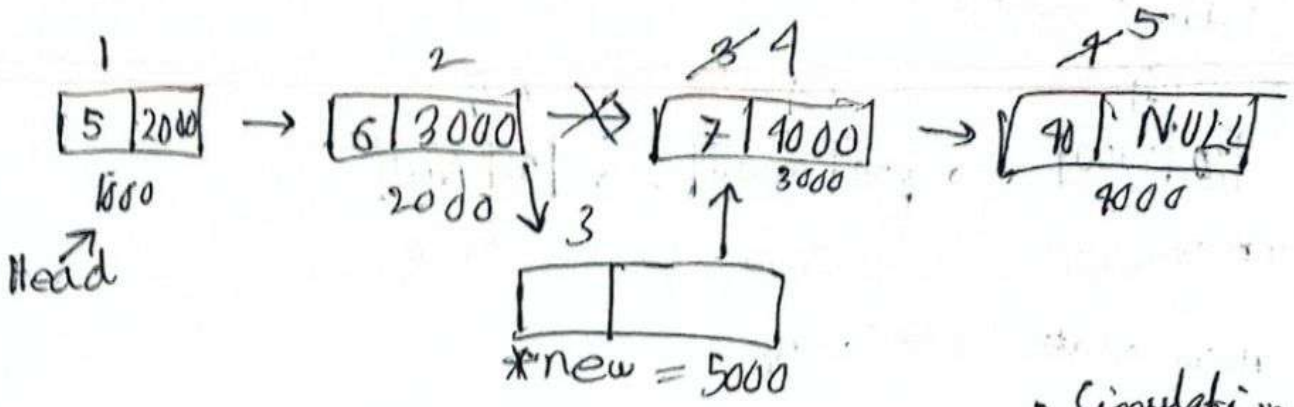
~~Algorithm~~: Insert at last




```
struct node *temp = head;
while(temp→next != NULL)
{
    temp = temp→next;
}
```

শেষ অবধি temp Last node এ আসবে

```
struct node *new = malloc(.....)
    new→data = value; //40
    new→next = NULL;
    temp→next = new;
```




```
struct node *temp = head;
int pos;
scanf("%d", &pos);
for (int i=1; i<pos-1; i++)
{
    temp = temp->next;
}
struct node new* = malloc(---)
new->data = value
new->next = temp->next
temp->next = new
```

important lines

Simulation

temp = 1000

Round 1

i = 1

i < pos-1

1 < 3-1

1 < 2

true

temp = 2000

R2

i = 2

2 < pos-1

2 < 3-2

2 < 2

false

for loop থেকে বের হয়ে যাবে

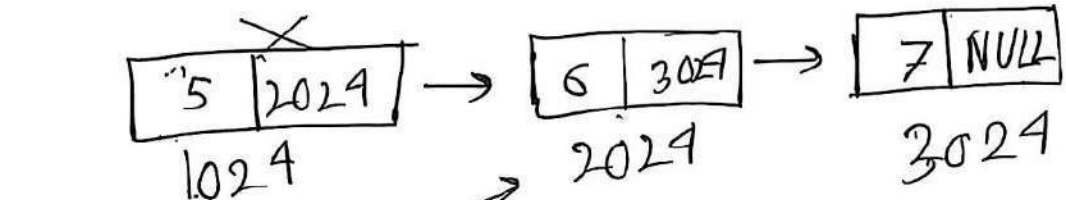
temp থাকবে 2000

01/10/26

free() function দিয়ে আমরা memory release করতে (Malloc এর উল্টা)

Deletion

Deletion from head



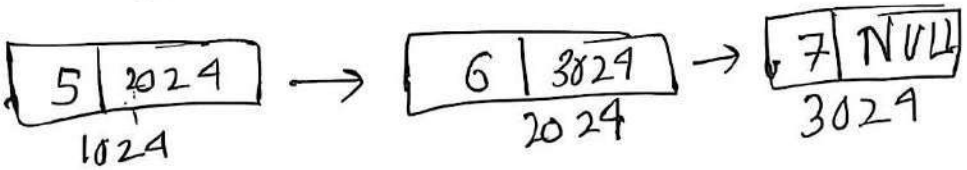
head

struct node *temp = head; //temp=1024

head = head -> next

free(temp);

Deletion from last -



head

```
struct node *temp = head, *prev;
while (temp->next != NULL) {
    prev = temp;
    temp = temp->next;
}
free(temp);
prev->next = NULL;
```

Simulation

temp = 1024

R1

2024 | =NULL
prev=1024;
temp=2024

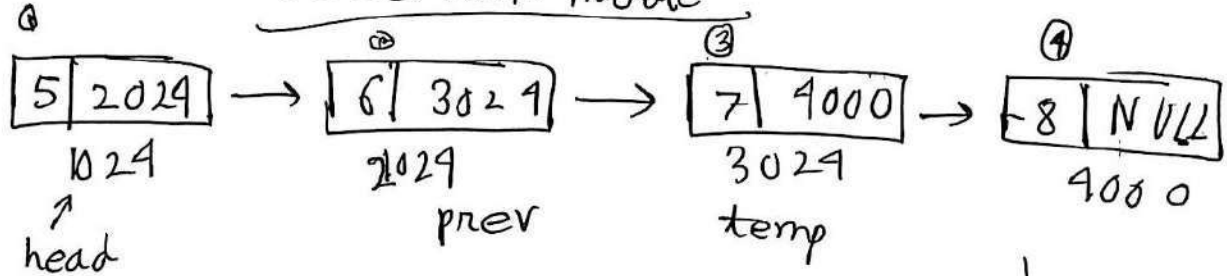
R2

3024 | NUL
prev=2024;
temp=3024

R3

NULL != NULL

Delete from middle



③ ka Delete karo

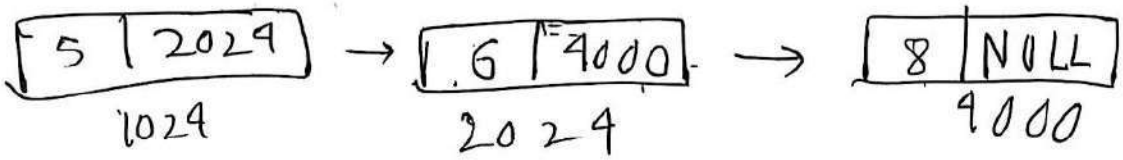
```
int pos;
scanf("%d", &pos); // 3
struct node *temp = head, *prev;
for(int i=1; i<pos; i++)
{
    prev = temp;
    temp = temp -> next;
}
prev -> next = temp -> next
free(temp);
```

temp = 1029
prev = ?

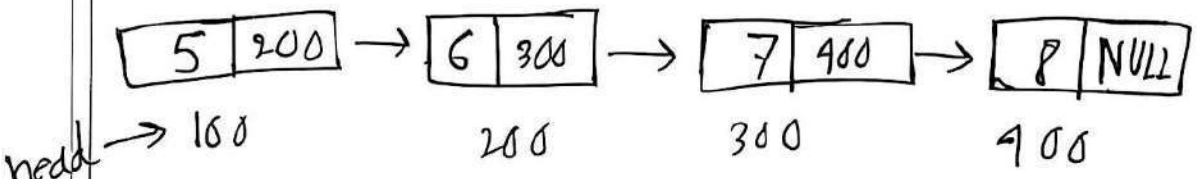
R1
i = 1
i < pos
1 < 3
prev = 1029
temp = 2029

R2
i = 2
2 < 3
prev = 2029
temp = 3029

R3
i = 3
3 < 3 ✗



Search



key = 8

```
int pos = 0
struct node *temp = head;
while(temp != NULL)
{
    pos++;
    if (key == temp->data)
    {
        printf("Item found in pos %d", pos);
    }
    temp = temp->next;
}
```

temp = 100

R1
100 != NULL
8 == 5
temp = 200

R2
200 != NULL
8 == 6
temp = 300

R3
300 != NULL
8 == 7
temp = 400

R4
400
NULL != NULL
8 == 8
temp = NULL

R5
NULL != NULL

Circular singly Linkedlist

```
struct node *temp = head;
while (temp -> next != NULL)
{ temp = temp -> next;
}
temp -> next = head;
```

7/10/2026

ODD No. Node ಇದ್ದ Linked list ದಲ್ಲಿ middle node delete

```
void deletmiddle()
{ struct node *temp = head, *prev;
  int n=0;
    while(temp != NULL) {
        n++;

        temp = temp -> next;

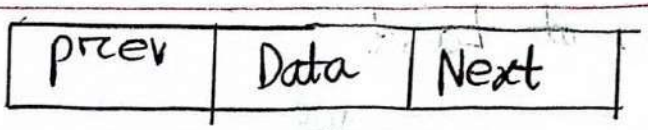
    }

  int m = n/2 + 1;
     temp = head;
  for(int i=1; i<m; i++)
  {
        prev = temp;
        temp = temp -> next;

  }
  prev -> next = temp -> next
  free(temp);

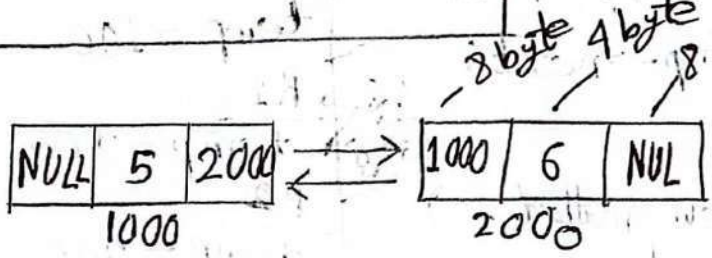
}
```

Doubly Linked List



Singly linked list – 12 byte
Doubly " " – 20 byte

```
struct node{
    int data;
    struct node *next;
    struct node *prev;
};
```



Initially *head = NULL

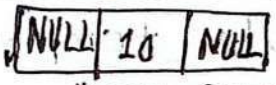
Insert at head

```
struct node *new = malloc(sizeof(struct node));
new->data = value;
new->next = head;
new->prev = NULL;
if(head != NULL){
    head->prev = new;
}
head = new
}
```

Simulation

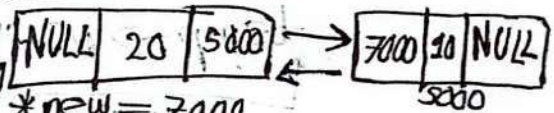
Initially
*Head = NULL

R1 ← Insert 1st time



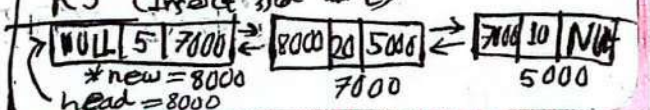
*new = 5000
head = 5000

R2 (Insert 2nd time)



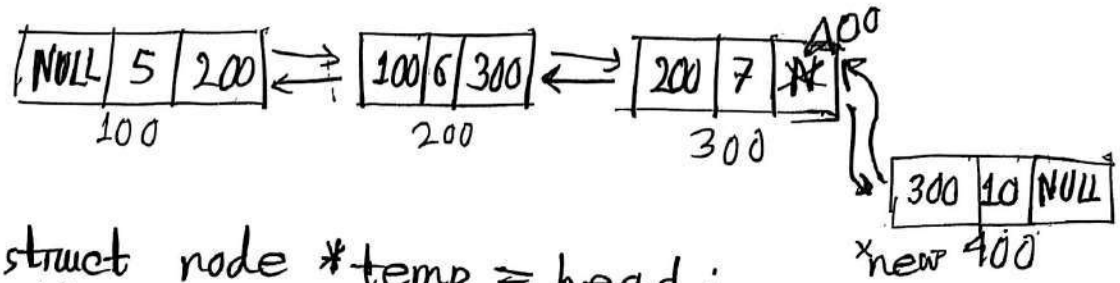
*new = 7000
head = 7000

R3 (Insert 3rd time)

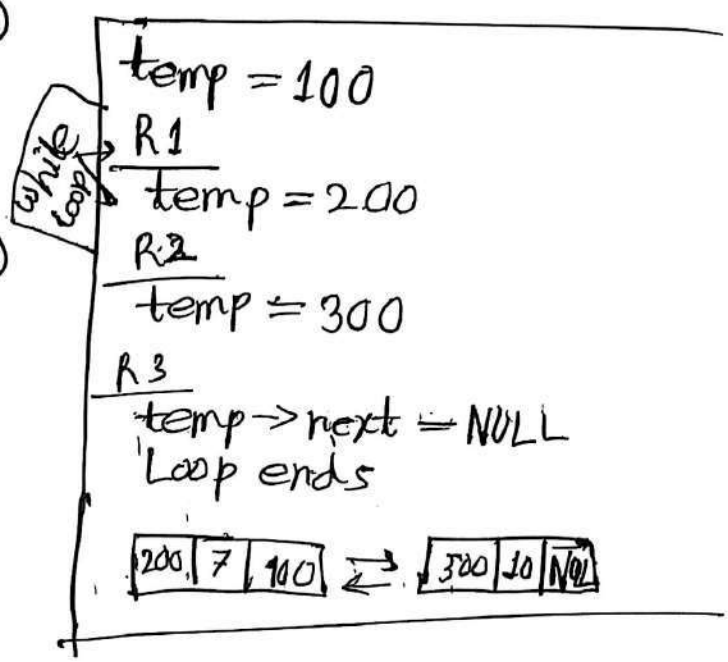


*new = 8000
head = 8000

Insertion at Last




```
struct node *temp = head;
while(temp->next != NULL)
{
    temp = temp->next;
}
struct node *new = malloc(..)
new->data = value;
new->next = NULL;
new->prev = temp;
temp->next = new;
```

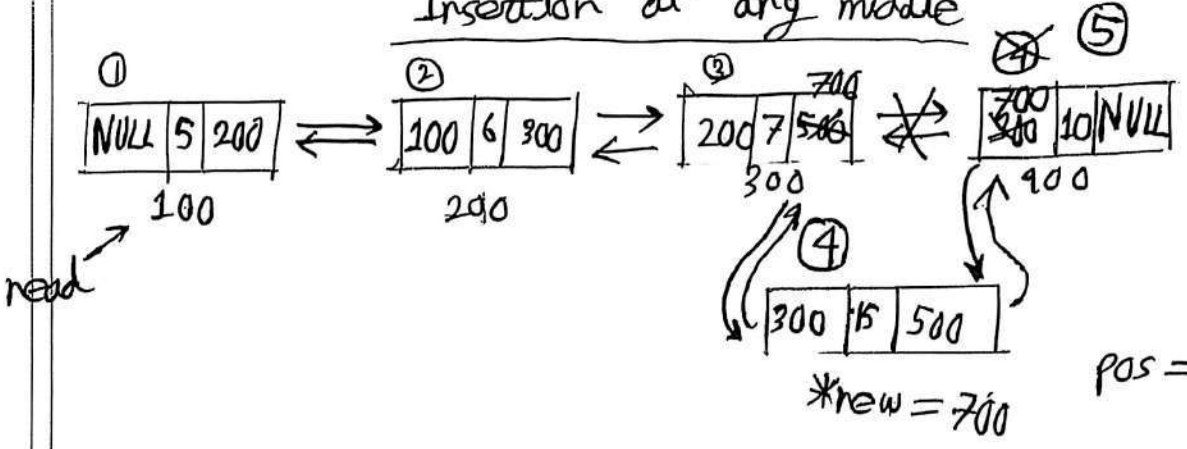


Delete at Last

```
struct node *temp = head;
while (temp->next != NULL)
{
    temp = temp->next;
}
temp->prev->next = NULL
free(temp);
```

300

Insertion at any middle




```
struct node *temp = head;
for (int i=1; i<pos-1; i++)
{   temp = temp -> next;
}
struct node *new = malloc(...)
new -> data = value;
new -> next = temp -> next;
new -> prev = temp ;
temp -> next = new ;
new -> next -> prev = new;
```

500

simulation

Loop ⓘ temp = 100

R1 ~~temp~~ i = 1

temp = 200

R2 i = 2

temp = 300

R3 ~~i < pos~~

i = 3

pos+1 = 4-1 = 3

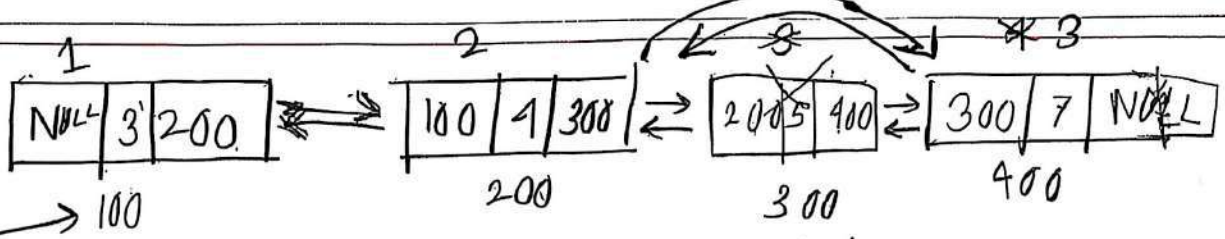
3 ✗ 3

Loop ends

∴ After loop, temp = 300

8/10/2026

Delete from any middle



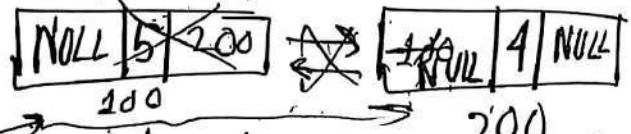
Head → 100

Delete(3)

```
struct node *temp = head;
for(int i=1; i<pos; i++)
{
    temp = temp->next;
}
temp->prev->next = temp->next;
temp->next->prev = temp->prev;
free(temp);
```

temp = 100
i = 1 < pos
R1 1 < 3
temp = 200
i = 2 < 3
R2
temp = 300
i = 3 ≮ 3

Delete from head

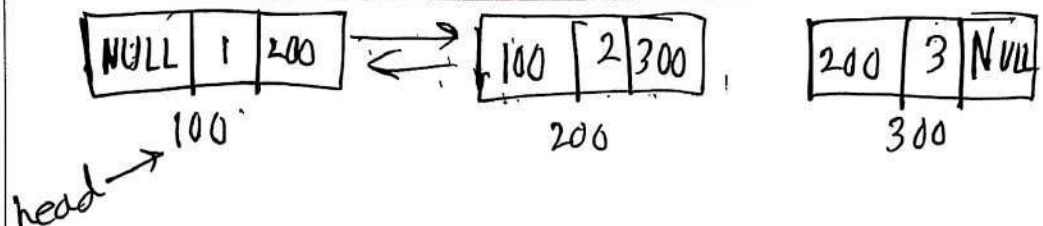


head →

```
struct node *temp = head; // temp = 100
head = head->next          // head = 200
head->prev = NULL;
free(temp);
```

Ring topology

Circular doubly linked list




```
struct node * temp = head;
while (temp -> next != NULL)
{ temp = temp -> next
}
temp -> next = head
head -> prev = temp
```

temp = 100
R1
temp = 200
R2
temp = 300

Last node ~~till~~ Loop

```
struct node * temp = head;
while (temp -> next != NULL){
        temp = temp -> next;
}
```

~~Ring topology~~

```
do {
printf("%d", temp->data);
temp = temp -> prev;
while(temp != head)
```

```
while (temp != head) {
printf("%d", temp -> data);
temp = temp -> prev;
}
printf("%d", head -> data);
```

Solution by mam

```
struct node *temp = head;
while (temp -> next != NULL)
{  temp  = temp -> next;
}

while (temp != NULL)
{ printf("%d", temp->data);
  temp = temp -> prev;
}
```

Time complexity

Delet, Insert at head O(1)

Delete, Insertion at End O(n)

Delete, Insertion at specific location (Middle) O(n)

Useful

Space – Array তে আগাম space বরাদ্দ থাকে আমরা বরাদ্দ করে Linked list ও dynamically আমরা নিজ দায়িত্বে মুক্তি

Online class - ৪ থেকে ৩টা ক্লাস (৩ ঘন্টা নিবে)

15-Oct-26 ১০ নাম্বার Must এসাইন

The right to Information act 2009

Setiaon - 2, 4, 6, 7, 8, 9, 24, 32 + Seedule

Setion2 [slide-3]

Section2 - [slid-5]

Information means any record or matrial - such as memos, books, maps, contracts, data, reports, accounts, photographs, audio, video, films, or any electronic or machine-readable documents - related to the structure, constitution ~~of an authority~~.

রাষ্ট্র vs নাগরিক

~~Sli~~

Setion 6 slide 7

~~authority এস~~

authority ও কাদেরকে কাকে কবে disclose করতে হবে।
annual report বলে ~~আলাদা~~ Report কে বলে। CSS এ
কোন গাছে কয়টা লিফ থাকবে
slide এর বাইরেও পড়া লাগবে।

Section 8 -

Section 9

Online Class

DSA

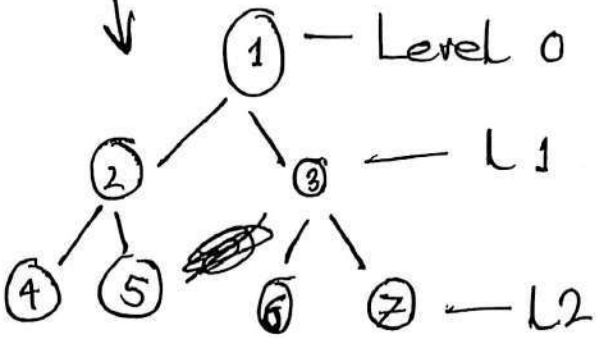
Click here to access the online DSA class of 17.10.2026

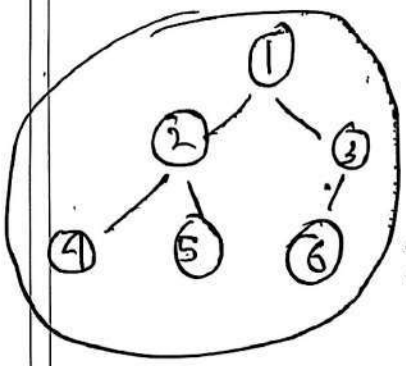
Click here to access the online DSA class of 31.10.2026

Online
17-10-26

Heap

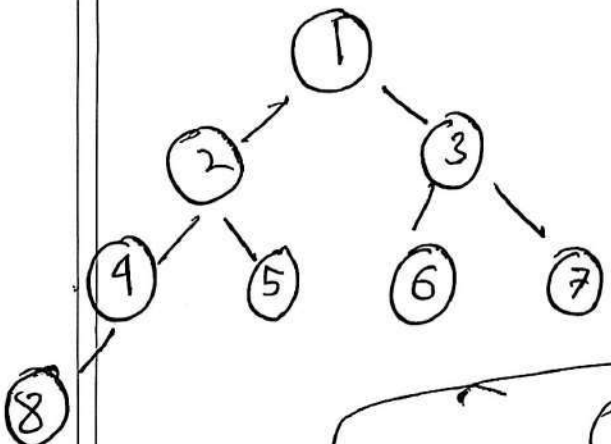
Compelete Binary tree (CBT)



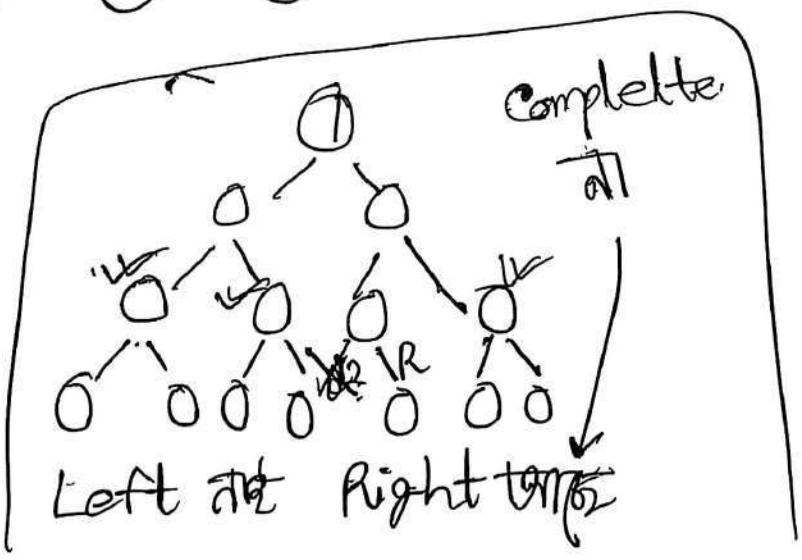


complelet

Last Level এ ডান দিকে কোনটা Absent থাকতে পারবে।

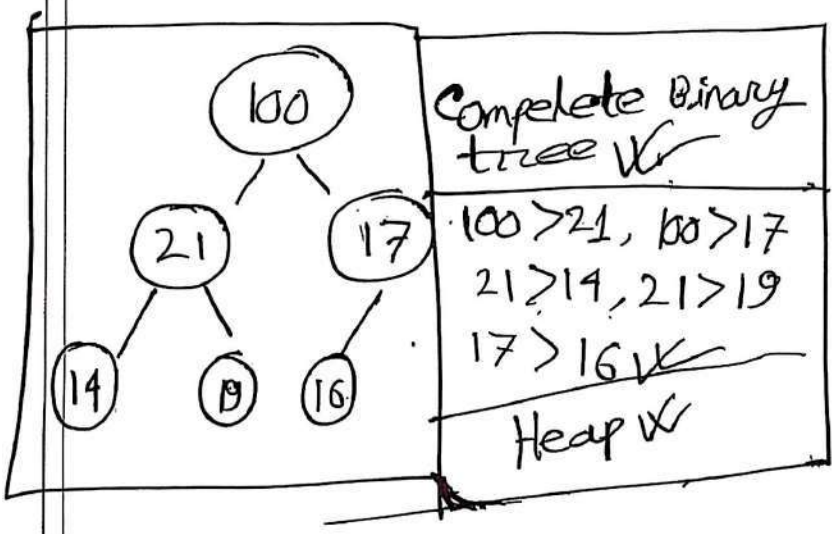


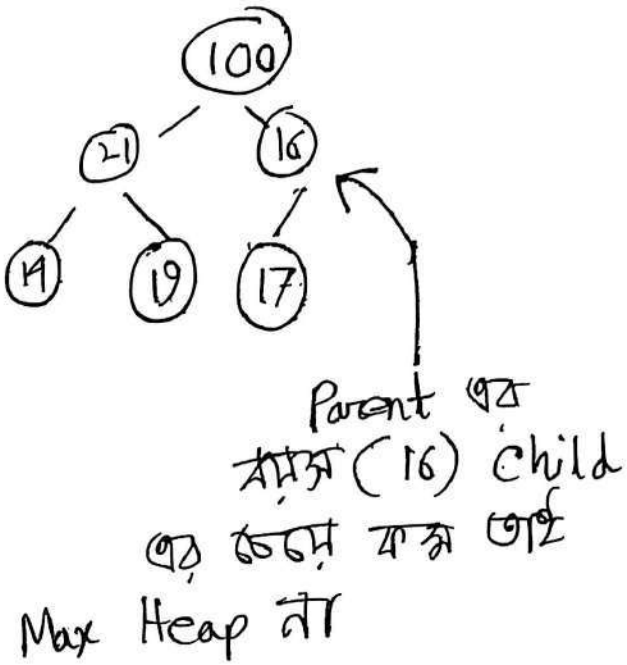
← Complete Binary tree



Heap হলো একটি Complete Binary tree

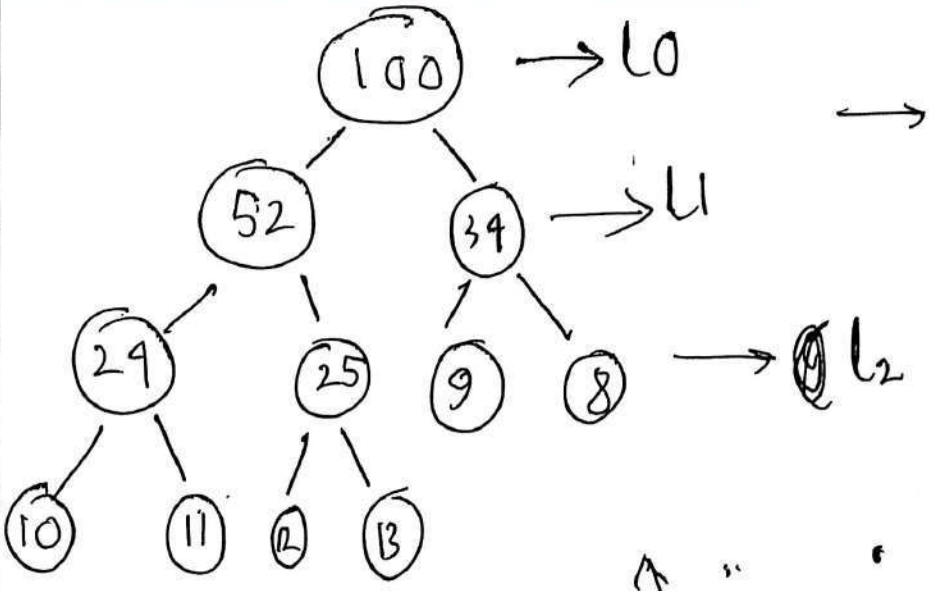
- Max heap — যেখানে Level এ Parent এর value child এর থেকে বড় বা সমান
- Min heap





Heap Operations

1. Insertion
2. DeleteMax or delete min
3. Make-heap / heapify

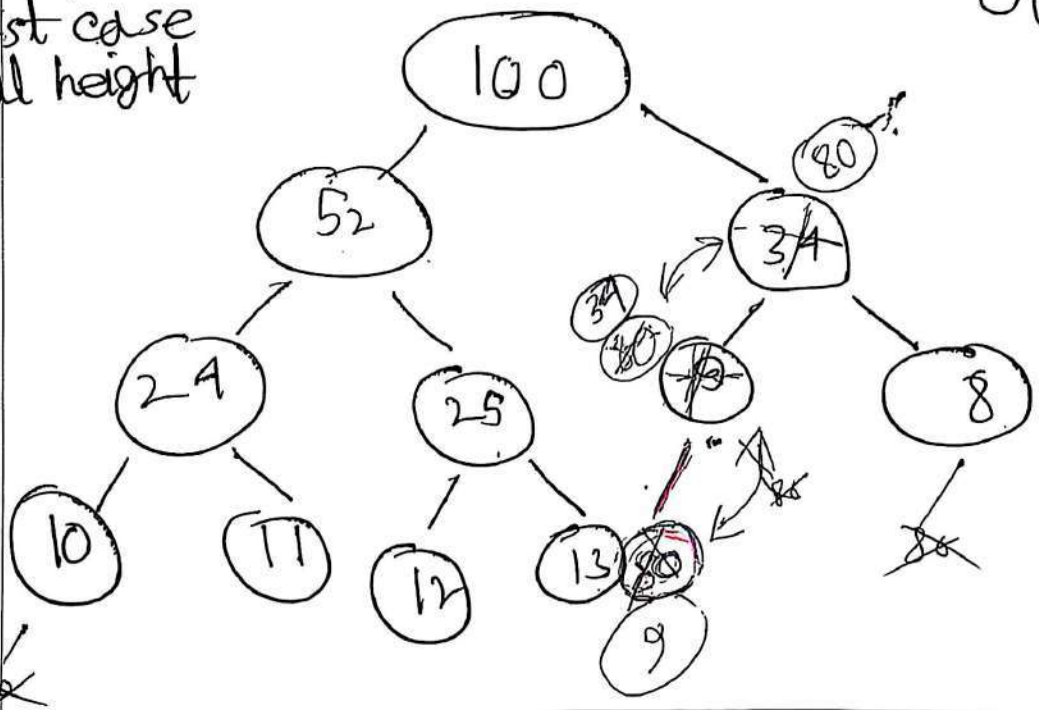


→ CBT ✓

Max-heapx

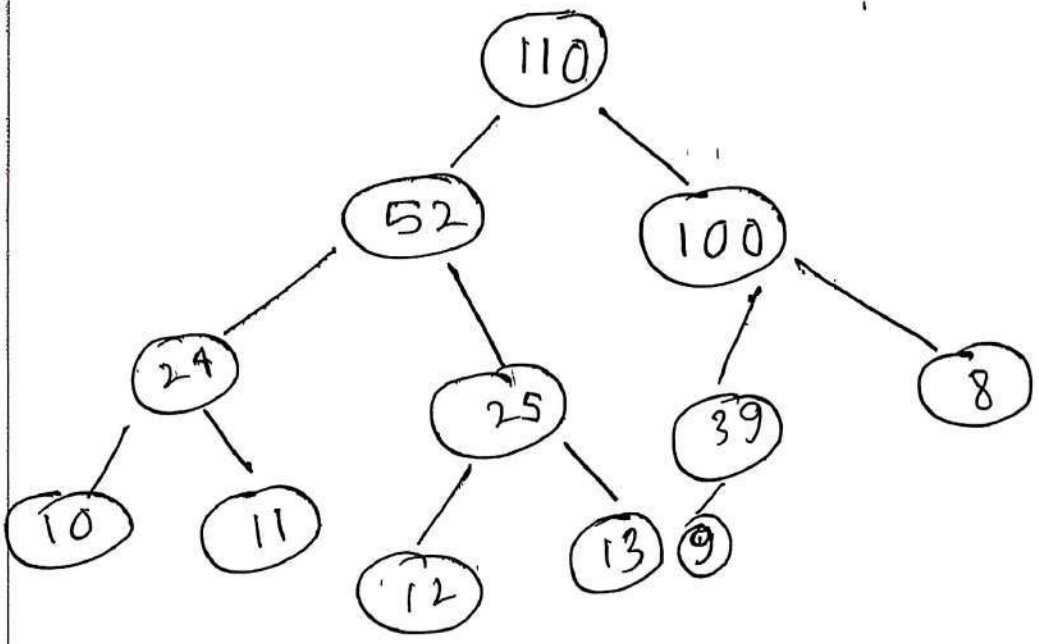
Insertion, ○ Insert, delete এর time complexity $O(\log_2 n)$

Worst case এ full height



Tree related যত আলোচনা Algorithm নিয়েই না। Simulation নিয়েও।

Randomly যেকোন সাধারণ থেকে Insert / Delete করা যায় না।



ঘ।

Heapify (Heap বানানো)

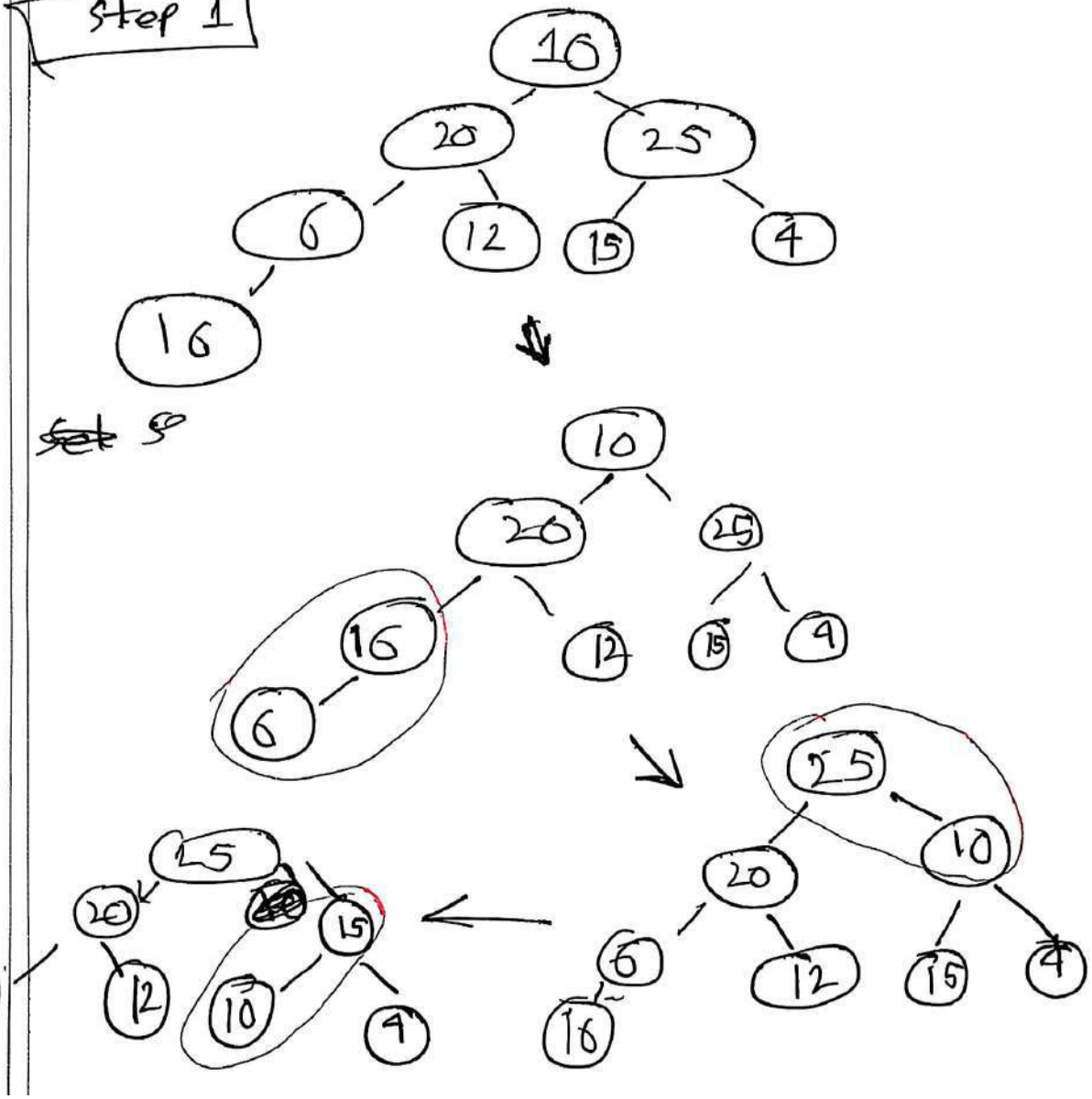
- Max heapify
- Min "

Step 1: যেই order এ আমাকে সেই order এ Heap বানাব

Step বাকিগুলো: swap - swap - swap - swap

10, 20, 25, 6, 12, 15, 4, 16

Step 1



Variant of Heap

1. Binary Heap
2. Binomial
3. Fibonacc
4. Pairing
5. Treap
6. Beap

Application

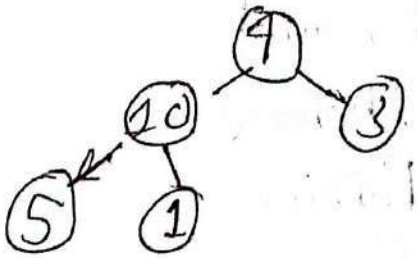
1. Heap Srt
2. Priority Q
3. Selection Algorithm
4. Graph algorithm

28/Oct/2026

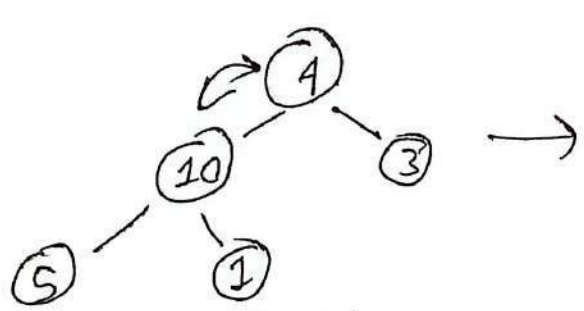
Heap.ppt (Slide 26)

Slide 27

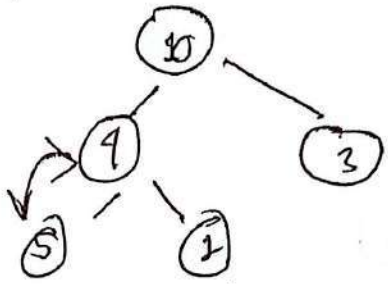
Arr = {4, 10, 3, 5, 1} →

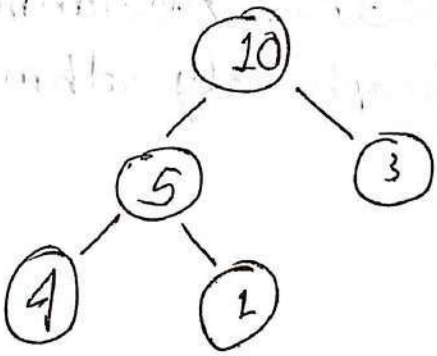


Transfer into max heap:



নিচের Level এ কোন swap প্রয়োজন হয় নাই

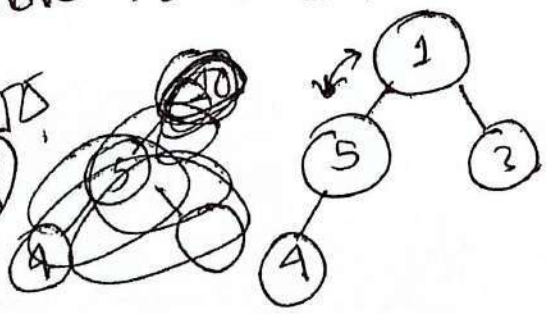


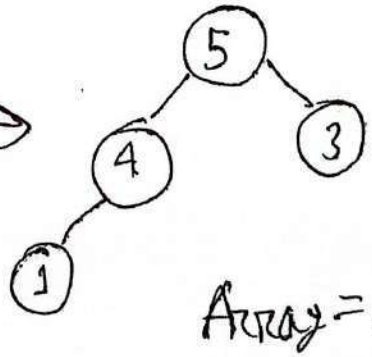


Slide 28

Remove Root (10)

(Right most element এর সাথে swap)

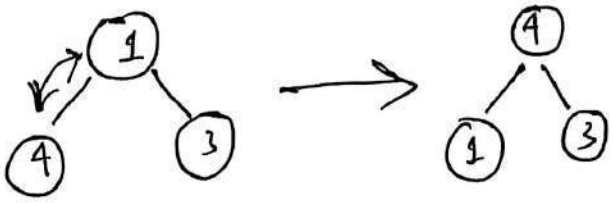




Array = {... 10}

Slide 30

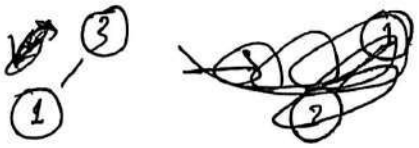
Remove Root (5)



Arr = {.... 5, 10}

~~Slide~~ Slide 31

Remove Root (4)



Arr = {...4, 5, 10}

~~Slide~~

Remove Root (3)



Arr = {..3, 4, 5, 10}

Arr = {1, 3, 4, 5, 10}

time complexity $O(n \times \log_2 n)$

Heap sort Traditional গুলার চেয়ে Faster. But Counting sort এর ক্ষেত্রে Range(k) না জেনে কোনটা Faster বলা যাচ্ছেনা।

Counting sort faster হবে যদি range খুব বেশি বড় হয়।

AD BREAK

Download your cover page and exam routine, see Utsob's notes, and much more.

Click here to visit Utsob's website



Binary Search Tree (BST)

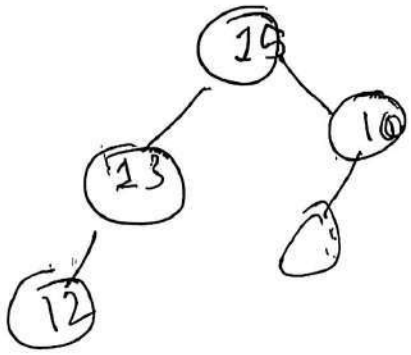
It is a hierarchical, nonlinear data structure

Left child, Parent এর চেয়ে ছোট হবে
Right child, Parent এর চেয়ে বড় বা সমান হবে

(সমান হলে যেকোনো একটি বামদিকে অথবা ডানদিকে রাখবেন)

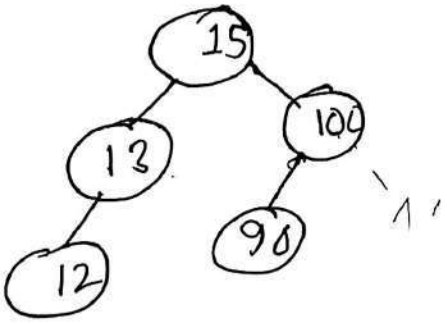
Search(Node, Key, Par)
1. If Node = NULL then return NULL
2. If Key < Node.key then search(Node, key, Node)

Tree এর কোন Algorithm লাগবে না সেহেতু লাগে পড়ার নাই।

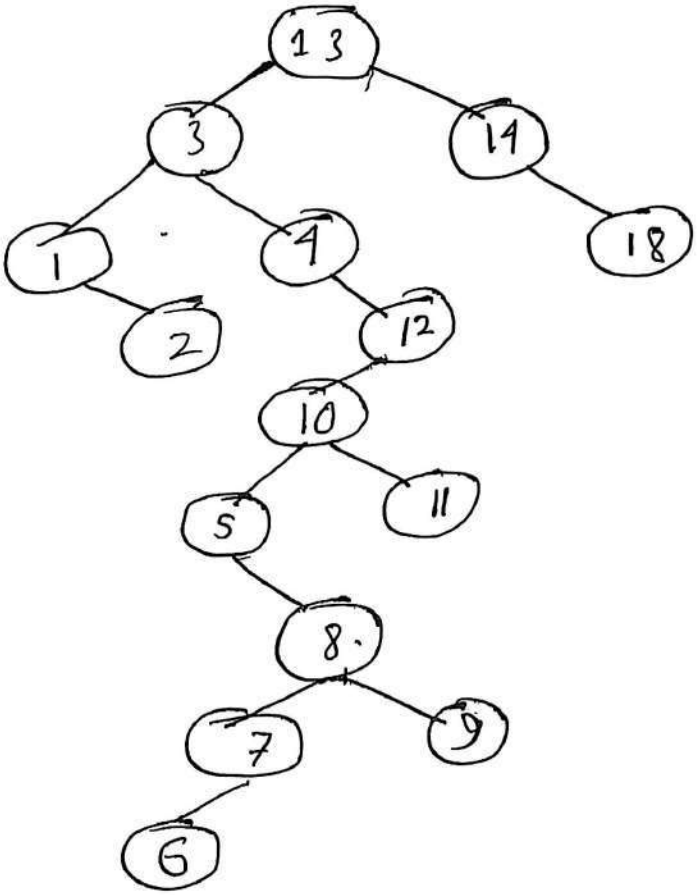


Insert

Insert (110)



- 13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18 in that order, create a BST starting from an empty tree

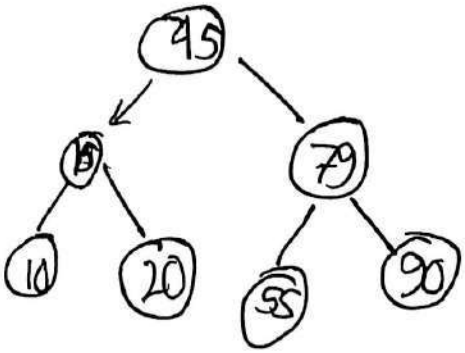


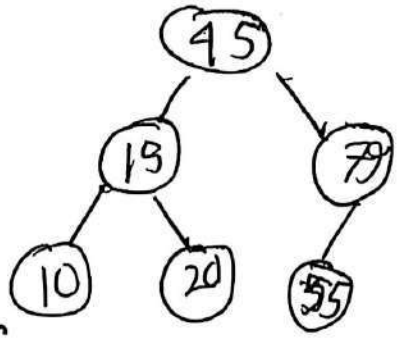
29/10/26

~~Inorder~~

Deleting a node from Binary search tree

delete(90)



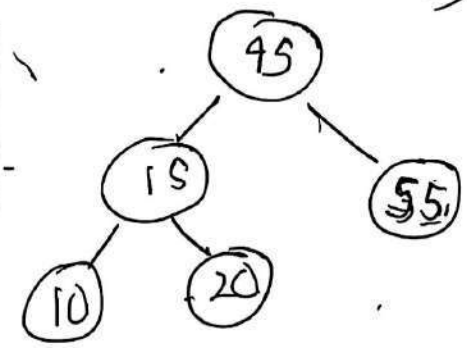


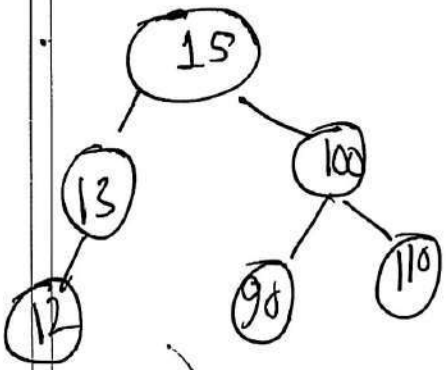
delete (45) next page

delete (79)

(Inorder successor)

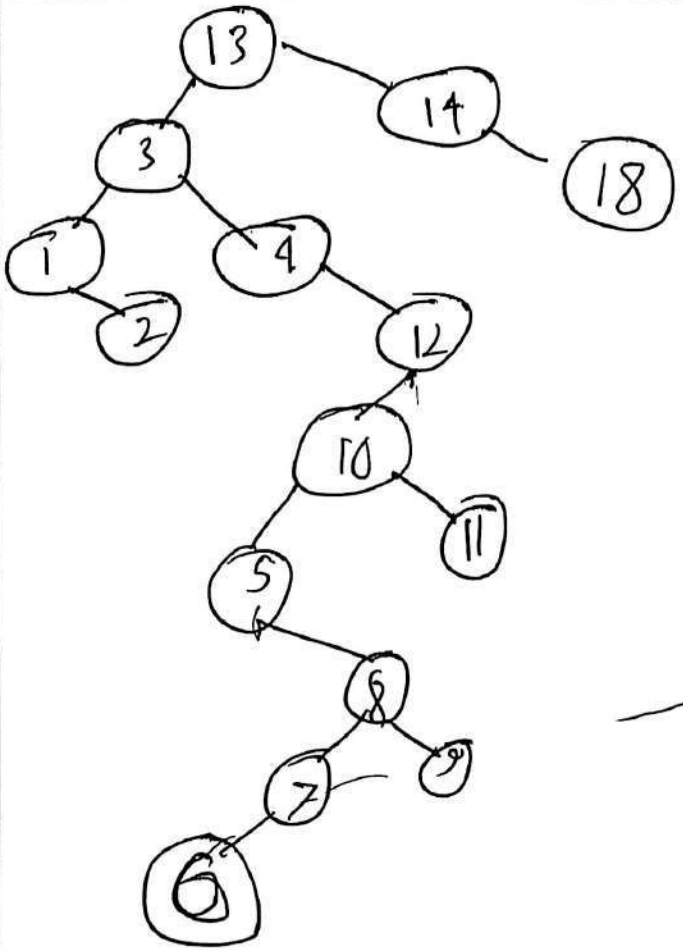
(ইনঅর্ডার সাকসেসর)



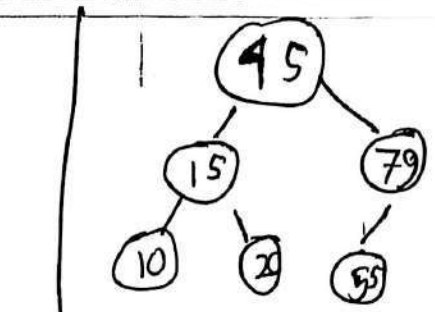


Inorder traversal : 12, 13, 15, 90, 100, 110

(Left → Root → Right)



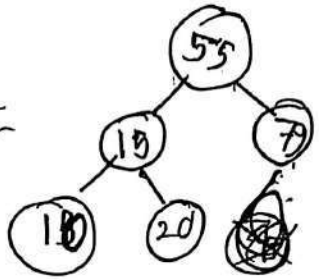
~~Inorder: 6, 7, 8, 9,~~



delete (45)

Inorder : 10, 15, 20, 45, 55, 79

পূর্ববর্তী পরবর্তী



Inorder: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 18,

(Ascending order এ আসা)

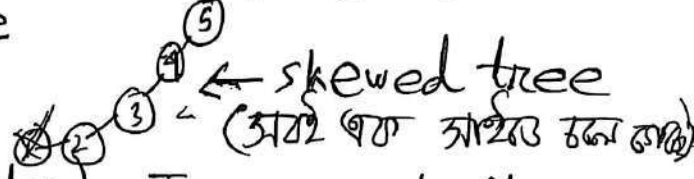
Time Complexity: $O(\log_2 n)$

↑

Insert, delete average case

delete worst case

example $O(n)$



← skewed tree

(সবই বাম সাইডে চলে গেছে)

BST

Insert, delete, Sort(Inorder), Time complexity

Tree এর Last topic

Huffman Algorithm

1952 সালে দিয়ে এসেছে Algorithm. (Compression)

Application:

- File compression (ZIP, GZIP)
- Image formats (JPEG, PNG)
- MP3 audio compression
- Network protocols

Slide 7

Example

apple□pie

Weight

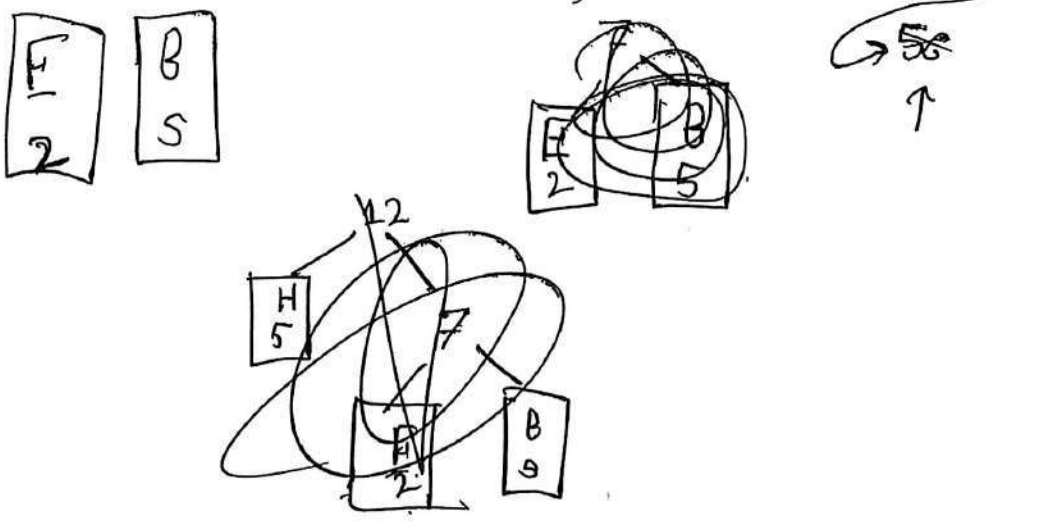
a : 1
p : 3
l : 1
e : 2
i : 1
space → □ : 1

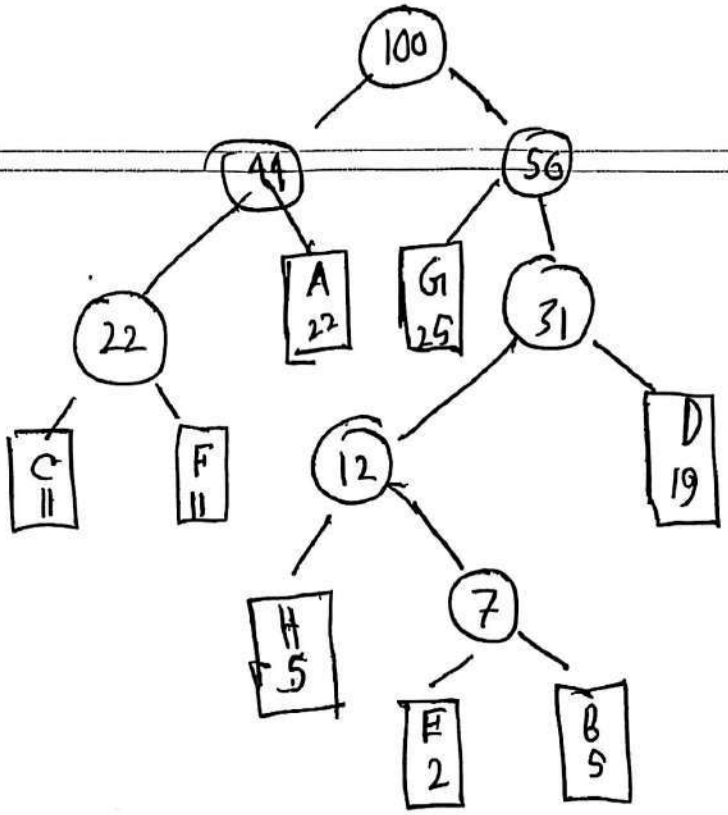
Given

A	B	C	D	E	F	G	H
22	5	11	19	2	11	25	5

প্রথমে Ascending আকারে সাজাতে হবে

E	B	H	C	F	D	A	G
2	5	5	11	11	19	22	25




100
44
56
A
22
G
25
22
31
C
11
F
11
12
D
19
H
5
7
E
2
B
5

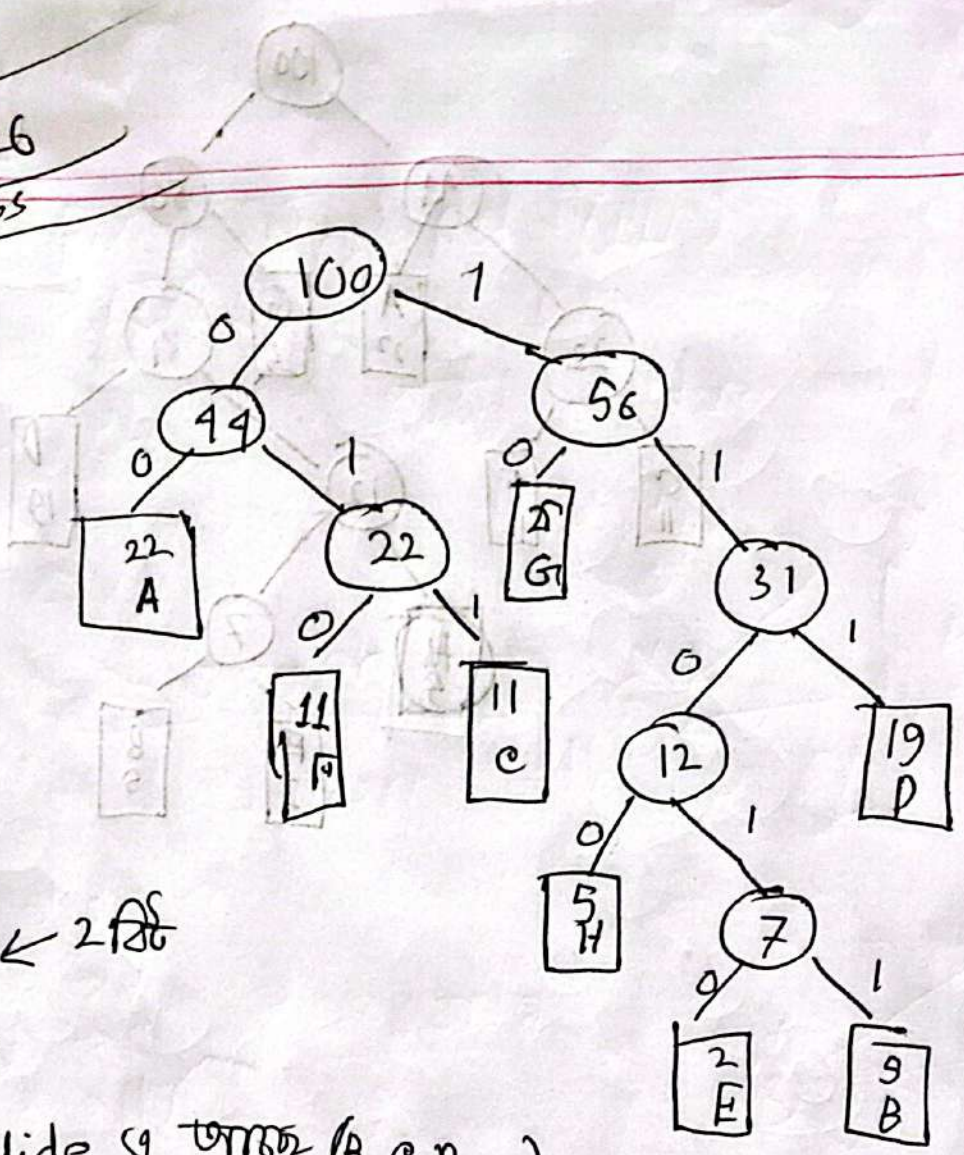
Online Class

DSA

Click here to access the online DSA class of 17.10.2026

Click here to access the online DSA class of 31.10.2026

31/Oct/2026
online class



A : 00 ← 2 বিট

F : 010

বাকিগুলা slide এ আছে (B, C, D ....)

ক্লাসেজ পাশ্চাত্য ABC

প্রচলিত হয় Traditionally A = 000000 ... ← 7 বিট

$= 3 \times 8 = 24b$

A = 00 – 2 b

B = 11011 – 5 b

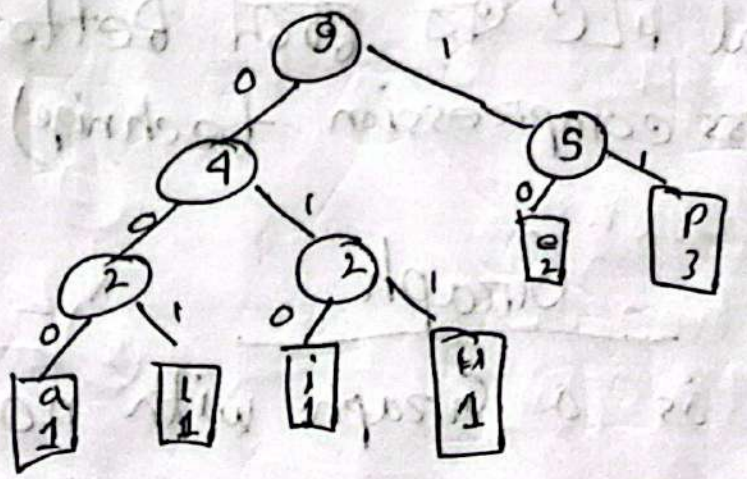
C : 011 – 3 b

Huffman দিয়ে পাঠালে 2 + 5 + 3 = 10 b যেই Normal 24 b লাগত

Haffman Code= Variable Length Code

apple pie - Normally লাগতো $9 \times 8 = 72$ bits

Haffman দিয়ে করলে



apple pie = 000 11 11 001 10 011 11 010 10

a p p l e A p i e

(22 bits)

Size বাচলো $(72 - 22) = 50$ bits.

$\log_2 9 \approx 3.2$

Fixed Length Code (FLC),

~~Step~~: সহজ।

Huffman, FLC থেকে অনেক better
(Lossless compression technique)

Graph

A tree is a Graph with no cycle

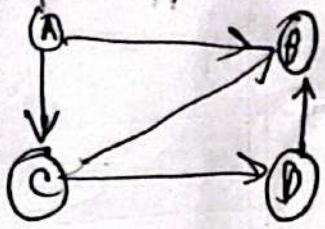
A → B → C

A → B

A → C

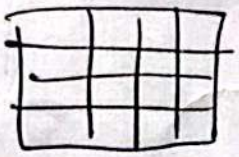
B → NULL

B → কুযাশা ন



Dense Graph vs Sparse Graph

Adcency list Representation, Adjacency matrix



Que: Directed or undirected matrix চিনবে
কুযাশা কয়ভাবে

Graph traversal

- Breadth-first search (BFS)
- Depth-first search (DFS)

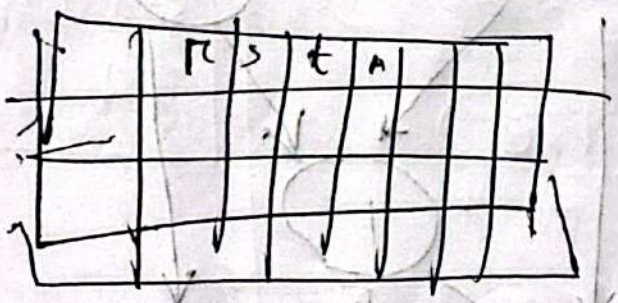
মডিউল আজকে পড়াবে কে পর্যন্ত CT

BFS (Core Man বই Chapter 22)

White — have been descoved
Gray — not fully ,,
White — fully ,,

BFS Example

Slide এ আছে



Level by Level explore করে

Q লাগে

Time — O(b+e)

4-Nov-2016

BFS, DFS দিয়ে tree কে traverse করা যায়

CSE 203: DFS-Depth first Search

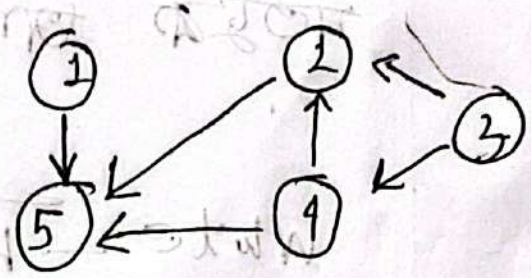
~~Sorsh~~ Source node দেওয়া থাকলে তো ভালোই কিন্তু না থাকলে তখন pic করা যায় randomly.

Input G (V, E).

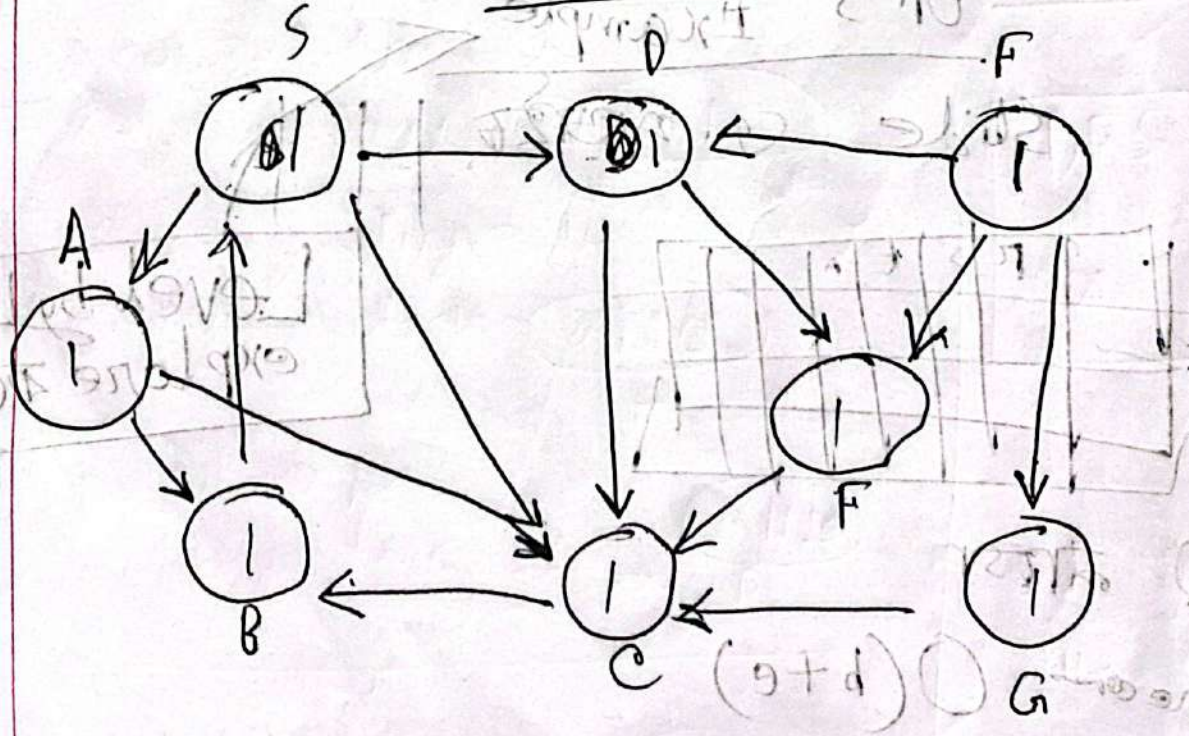
discovery time d[u]

finishing time f[u]

prev [u]



Example



Simulation slide ৭ আছে

Inisially সবগুলা color while (Undiscoverd)

S কে নিয়ে কাজ শুরু করলাম Gray হইল
শুরু হইছে কোন শেষ হয় নাই

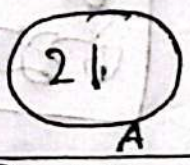
S কে দিয়ে শুরু করছি তাই এর কোন Parent ~~আসার possible~~ না নাই

discovery time d s f ← finishing time



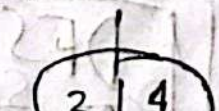
finishing
discovery time হবে
last time এর আগের 1মোগ

Next A কে discover করব। A হবে এখন gray



color = Gray
prev = S

Next B কে নিব। B = gray
prev = A



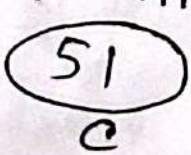
B এখন gray (S) কে point করে।
তাই ~~এর~~ দিকে যাব না।

prev (A) এর কাছে যাব। B.color = black (finished)

B এর ~~time~~ finishing time হবে 3+1=4

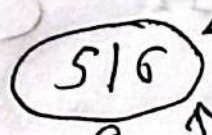
A তে আছি।
A এর একটা undiscoverde vertex C আছে।

C তে যাব



← gray

C এর আর কোন child নাই যাওয়ার মত



↑ finishing time = 6
color = black

A তে Back
finishing time 7
color = black → 2|7
A

S এ আসছি।
আর child D নাই।

D তে আসছি
discovery time 8
color = gray → 8|
D
prev = S

E তে আসছি
discovery time 9
color = Gray → 9|
E

finishing tirge 10
color = black → 9|10
E

D তে আসছি
finishing tine 11
color black → 8|11
D

S এ আসছি
finishing time 12
color black → 1|12
S

BFS tree এর উদাহরণ কাজে করবে

DFS forest এর উদাহরণ কাজে করবে

F আর G বাকি রইল

একটা tree explore done (last time = 12)

নতুন tree এখন

এখন যেকোনো একটা (F বা G) থেকে যেতে পারব।

F এ আসলাম

discovery time = 13 → (13|) F

color gray

G তে এ আসলাম

discovery = 14 → (14|) G

color gray prev = F

G' তে আর white child নাই

finishing time = 15

color = black → (14|15)

F এ আসলাম

finishing time = 16

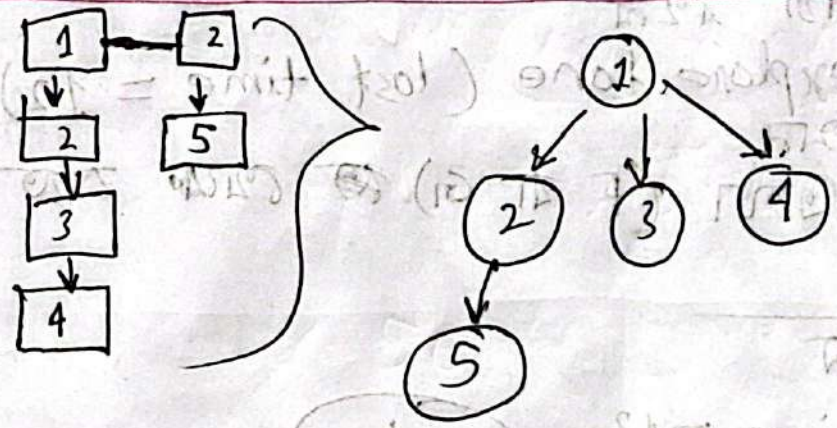
color = black → (13|16)

All node done

প্রশ্নে কোথাও বলা থাকেনা DFS নাকি BFS

Adjacency matrix, adjacency list

Data structure এর নাম Graph
Algorithm এর নাম DFS, BFS



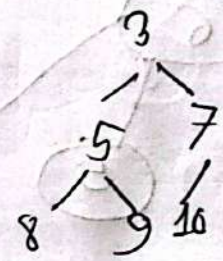
& DFS uses Stack

DFS.pdf তে আছে।

gray হলে push
black " pop
back track এ আসে.

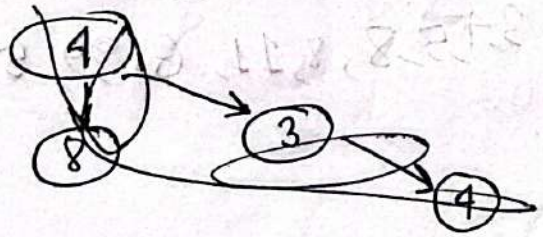
Hafman algorith [Lossless compression
variable length code
fixe length code $\log_2$ length

1. Define min-heap with an example
⇒ A min heap is a compelete binary tree where the value of each node is less than or equal to the values of its children.
Example:



2. Deive time complexity of heap sort
Building heap from initial array requires $O(n \log_2 n)$
Maximum the delete operation will perform n times
∴ time complexity $= n * O(\log_2 n) = O(n \log_2 n)$

3. ⇒
Let $x = 56$ (chaade choose করবো)



Knowledge
2% 6 = 2
56% 6 = 2
6% 6 = 0
7% 6 = 1
8% 6 = 2